

Appunti Sistemi Operativi – LAB Parte_2

Sommario

Capitolo 7 – Gestione dei Processi	2
Introduzione	2
Funzioni di uscita	2
Funzione atexit	3
Environment List	3
Struttura della memoria per un programma C	4
Allocazione della memoria	4
Controllo dei Processi	5
System call fork()	7
Identificativi di Processo	9
Chiamata di sistema vfork()	13
Terminazione di processi in Unix	14
Chiamate di sistema wait() e waitpid()	15
Chiamata exit()	16
Processi zombie	16
Processi adottati	17
Race Conditions	17
Famiglia exec	17
Le varianti della famiglia exec	17
Relazione tra le funzioni della famiglia exec	19
Schema di chiamate a fork, exec e wait (waitpid)	19
Esempio: myexec.c	20
Esempio: myexec1.c	20
Esecuzione dei comandi nella shell	21
Esempio: esecuzione di un comando	21
Esecuzione di una pipeline da shell	22
Liste di comandi	23
Esempio: modello semplificato di shell	23
Esempio: creazione e attesa processi	24
Capitolo 8 – Segnali	25
Introduzione	25
La funzione signal()	25
Gestore dei segnali	27
Esempio: Gestore che “cattura” vari segnali e ne stampa il tipo	27
System call kill() e raise()	28
Richiedere un segnale di allarme: Funzione alarm()	29
Attendere un segnale: Funzione pause()	29
Funzione sleep()	30
Funzione nanosleep()	30
Funzione abort()	31
Segnali inaffidabili	31

Rientranza delle funzioni.....	32
Esempio	33
Alcune Funzioni rientranti.....	33
Esempio d'uso dei segnali	34
Esempio: critical.c.....	35
Esempio: limit.c	36
Esempio: pulse.c (sospensione e ripresa)	38
Segnali affidabili	39
Maschera dei segnali: sigprocmask()	39
Esempio	41
Evitare le race condition: sigaction().....	42
Esempio	43
Segnali per il controllo dei job.....	44
Gruppi di processi.....	44
Cambiare il gruppo: setpgid().....	44
Ottenere il gruppo: getpgid()	45

Capitolo 7 – Gestione dei Processi

Introduzione

Il **ciclo di vita** di un programma C ha inizio con l'esecuzione di una **routine di avvio** predisposta dal **compilatore**, nota come **start-up routine**. Questa fase iniziale ha il compito di **preparare l'ambiente di esecuzione**, recuperando le **variabili d'ambiente** e gli **argomenti** passati da riga di comando (contenuti nel vettore **argv[]**) e, una volta completata, invoca la funzione **main**.

La funzione **main**, tipicamente definita come `int main(int argc, char *argv[])`, riceve due parametri fondamentali: **argc**, che rappresenta il **numero di argomenti**, e **argv[]**, un **array di stringhe** che li contiene. Secondo le specifiche **ISO C** e **POSIX**, `argv[argc]` è garantito essere un **puntatore a NULL**, facilitando l'iterazione sugli argomenti. Il primo elemento, **argv[0]**, solitamente contiene il **nome del programma stesso**.

Alla conclusione dell'esecuzione, la **terminazione del processo** può avvenire in modo **normale** o **anomalo**. Nella **terminazione normale**, il programma può terminare restituendo il controllo al sistema operativo tramite il **ritorno dalla funzione main** oppure attraverso una chiamata esplicita alla funzione **exit**. Quando viene invocata **exit**, il sistema segue una **sequenza ordinata di operazioni**: vengono prima eseguiti gli **exit handler** eventualmente registrati con **atexit**, poi si procede con la **pulizia degli stream di I/O** aperti (ad esempio tramite **fclose**), e infine il controllo torna al **kernel** mediante una chiamata a **_exit** o **_Exit**, che concludono definitivamente il processo.

In caso di **terminazione anomala**, invece, il processo viene interrotto in seguito a eventi eccezionali, come la chiamata a **abort**, la ricezione di un **segnale** non gestito, o una **richiesta di terminazione** da parte dell'ultimo thread attivo. Questi scenari, non previsti nel normale flusso di esecuzione, comportano l'interruzione immediata e non ordinata del programma.

Funzioni di uscita

Le **funzioni di uscita** sono utilizzate per terminare un programma in modo controllato, restituendo un **codice di stato** al sistema operativo.

```
#include <stdlib.h>

void exit (int status)
void _Exit(int status)

#include <unistd.h>
void _exit(int status)
```

Queste tre funzioni **terminano normalmente** un programma. Tutte prendono in input un intero **status** che indica se il programma è terminato correttamente (valore 0) oppure no (valore diverso da 0). La funzione **_exit()** è una system call mentre la funzione **_Exit()** è una funzione della libreria standard ed entrambe chiudono immediatamente il programma. La funzione **exit()** fa parte della libreria standard e prima di chiudere il programma esegue delle operazioni di **pulizia degli I/O utilizzati**.

Lo stato di ritorno di un programma è **indefinito** se: non viene passato il codice di uscita, non viene indicato nessun valore al return oppure la funzione main non è dichiarata per restituire un intero.

Restituire un **valore intero** dalla funzione main, ad esempio tramite `return 0;`, è funzionalmente equivalente a chiamare esplicitamente `exit(0);`. Entrambi i metodi indicano al sistema operativo che il programma si è concluso con successo.

Funzione	Header	Standard	Esegue <code>atexit()</code> ?	Svuota buffer I/O?	Uso tipico
<code>_exit()</code>	<code><unistd.h></code>	POSIX	✗ No	✗ No	Terminazione immediata in ambiente UNIX (es. dopo <code>fork()</code>)
<code>_Exit()</code>	<code><stdlib.h></code>	ISO C99	✗ No	✗ No	Terminazione immediata portabile (standard C)

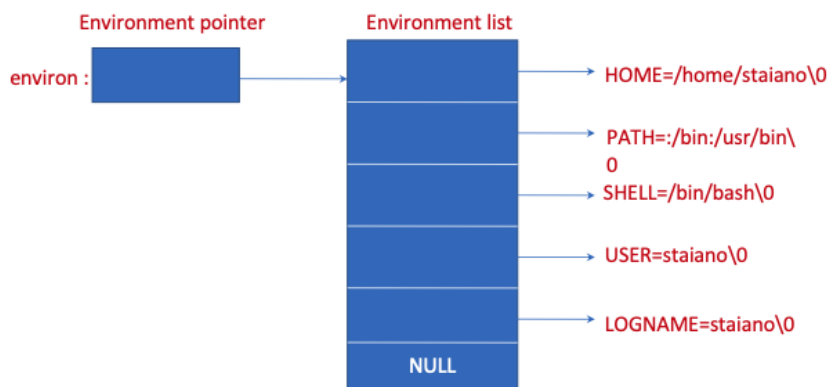
Funzione atexit

```
#include <stdlib.h>
int atexit (void(func*)(void));
```

La funzione **atexit** permette di registrare delle funzioni, chiamate **exit handler**, che verranno eseguite automaticamente alla **terminazione del programma** tramite `exit`. Lo standard **ISO C** garantisce la possibilità di registrare almeno 32 di queste funzioni, che non devono ricevere argomenti né restituire valori. La registrazione avviene passando ad `atexit` il **puntatore alla funzione** da eseguire, e il suo valore di ritorno indica se la registrazione è avvenuta con successo.

Le funzioni registrate vengono eseguite in **ordine inverso** rispetto a quello di registrazione. In conformità agli standard **ISO C** e **POSIX**, `exit` esegue prima gli **exit handler** e solo in seguito chiude gli **stream I/O** eventualmente aperti.

Environment List



L'ambiente corrisponde alla **configurazione** che il kernel mette a disposizione del processo in esecuzione (viene passato anche ai processi figli). Questo corrisponde ad un **array di puntatori a stringhe** (terminano con `\0`) il cui ultimo elemento è `NULL`. L'indirizzo dell'array di puntatori è contenuto in `extern char **environ` (storicamente si passava come terzo parametro al `main`) e le stringhe sono nel formato **nome=valore** (es. `HOME=/home/uni\0`).

Struttura della memoria per un programma C

La **struttura della memoria** di un programma C è organizzata in segmenti distinti, ciascuno con una funzione specifica, e segue una disposizione convenzionale che va da indirizzi bassi a indirizzi alti.

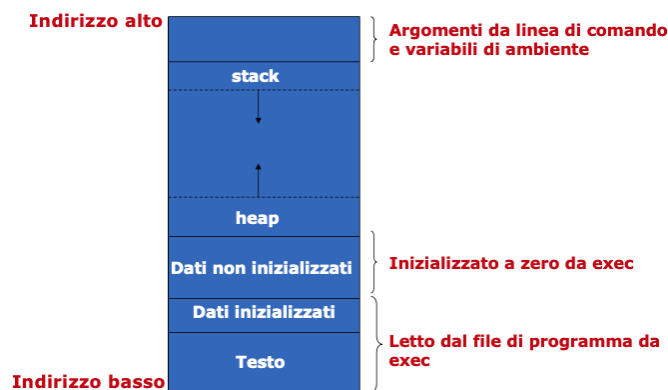
Il **segmento di testo** contiene le **istruzioni macchina** del programma, eseguite dalla CPU. Questo segmento è **condivisibile** tra più processi e di norma è impostato come **sola lettura** per evitare modifiche accidentali.

Il **segmento di dati inizializzati** ospita le **variabili globali o statiche** che hanno un valore iniziale assegnato nel codice. Ad esempio, una variabile come `int maxcount = 99;` risiede in questo segmento.

Il **segmento di dati non inizializzati**, o **bss** (block started by symbol), contiene variabili globali o statiche che **non sono inizializzate** nel codice. Queste vengono automaticamente poste a zero o a valori nulli dal kernel al momento dell'avvio del processo. È il caso, ad esempio, di `long sum[1000];`.

Lo **stack** è una struttura dinamica che gestisce le **variabili automatiche**, come quelle locali alle funzioni, e salva informazioni critiche ogni volta che viene effettuata una chiamata di funzione, come l'indirizzo di ritorno e i registri. Ogni nuova funzione alloca spazio sullo stack per i propri dati temporanei.

L'**heap**, invece, è lo spazio destinato all'**allocazione dinamica della memoria**, come avviene con le funzioni `malloc` e `free`. Questo spazio si colloca tra la zona `bss` e lo stack, crescendo in direzione opposta rispetto a quest'ultimo.



L'illustrazione della struttura evidenzia la disposizione sequenziale dei segmenti in memoria: dal basso verso l'alto si trovano il **testo**, i **dati inizializzati**, i **dati non inizializzati**, l'**heap** e lo **stack**, che termina con gli **argomenti da linea di comando** e le **variabili d'ambiente**.

Infine, il comando **size** consente di ottenere la dimensione (in byte) dei segmenti principali (`text`, `data`, `bss`) per un eseguibile. I valori vengono espressi anche in formato **decimale** e **esadecimale**, rendendo possibile un'analisi della **distribuzione della memoria** nel programma compilato.

Allocazione della memoria

```
#include <stdlib.h>
void *malloc(size_t size);
void *calloc (size_t nobj, size_t size);
void *realloc (void *ptr, size_t newsize);

void free (void *ptr);
```

L'**allocazione dinamica della memoria** in C è regolata da tre funzioni principali definite dallo standard **ISO C**: **malloc**, **calloc** e **realloc**. La funzione **malloc** riserva una quantità specificata di byte, **calloc** alloca spazio per un certo numero di oggetti inizializzandoli a zero, mentre **realloc** permette di modificare la dimensione di un blocco precedentemente allocato. A queste si aggiunge la funzione **free**, usata per liberare la memoria precedentemente riservata. Tutte queste funzioni restituiscono un **puntatore non nullo** se l'operazione va a buon fine, altrimenti **NULL**.

Le chiamate a queste funzioni sono implementate a basso livello tramite la system call **sbrk**, che serve ad **espandere o contrarre l'heap** del processo. Le implementazioni delle routine di allocazione spesso riservano più memoria di quella richiesta, per memorizzare metadati come la **dimensione del blocco** e un **puntatore al blocco successivo**, facilitando così la gestione interna della memoria.

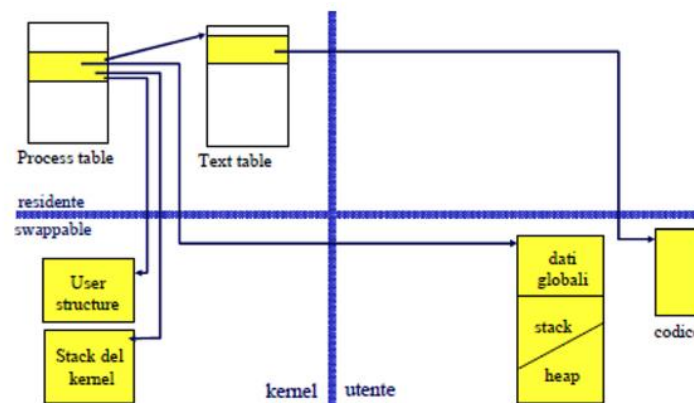
L'uso scorretto dell'allocazione dinamica può portare a **errori gravi**. Un primo rischio è scrivere oltre la fine di un'area allocata, corrompendo la memoria del blocco successivo. Un altro errore comune è liberare due volte lo stesso blocco con **free**, oppure usare **free** su un puntatore non ottenuto da **malloc**, **calloc** o **realloc**. Infine, dimenticare di chiamare **free** dopo aver riservato memoria porta a **perdite di memoria** (memory leak), con una crescita continua dell'uso di memoria da parte del processo.

Controllo dei Processi

Nel sistema operativo **Unix**, il **controllo dei processi** costituisce un elemento centrale nella gestione dell'esecuzione dei programmi. Unix è una famiglia di sistemi **multiprogrammati** che si fonda sull'utilizzo dei **processi** come unità fondamentali di esecuzione.

Ogni processo rappresenta un'entità **dinamica** dotata di uno **stato evolutivo**, che riflette il progresso nell'esecuzione di un programma. A differenza dei thread, i processi sono detti **pesanti** poiché ciascuno dispone di uno **spazio di indirizzamento privato** per i dati, non condivisibile direttamente con altri processi.

La **comunicazione tra processi** avviene mediante meccanismi espliciti, come lo **scambio di messaggi**, mentre la condivisione del codice è possibile grazie alla gestione **rientrante** del segmento di testo, condivisibile tra più processi senza conflitti, attraverso la **text table**.



Nel sistema Unix, l'**immagine di un processo** rappresenta l'insieme delle **aree di memoria** e delle **strutture dati** utilizzate durante l'esecuzione di un programma.

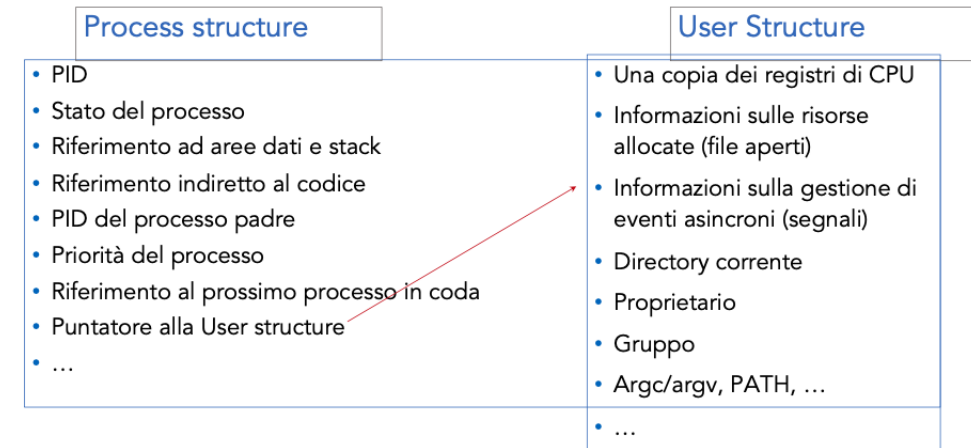
Essa è suddivisa in due parti principali:

- una parte **residente** in memoria centrale, che rimane allocata stabilmente,
- una parte **swappable**, che può essere temporaneamente trasferita in **memoria secondaria** per ottimizzare l'uso della RAM.

Le componenti accessibili dallo **spazio utente**, come lo **stack**, l'**heap** e i **segmenti dati**, sono tutte **swappabili**, poiché non indispensabili per la gestione diretta da parte del kernel.

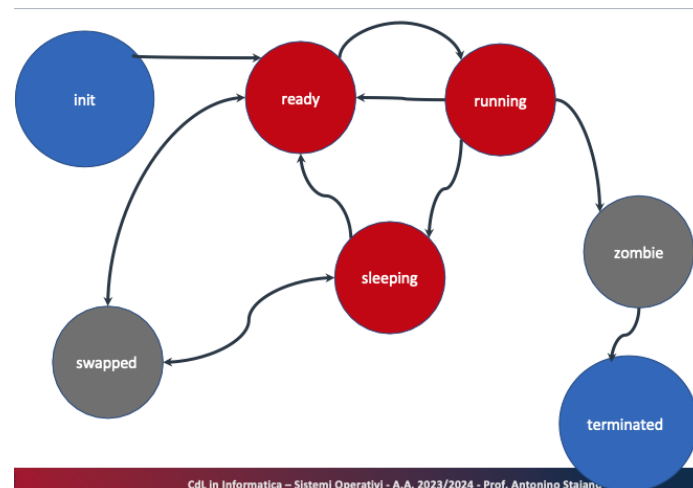
Le componenti lato **kernel**, invece, sono cruciali per il controllo del processo e includono la **process table**, la **process structure**, la **user structure** (u-area), lo **stack del kernel** e la **text table**.

La **process table** è una **struttura dati globale residente** in memoria centrale (non swappabile), accessibile solo in modalità kernel. Essa contiene una voce per ogni processo attivo nel sistema, e ciascuna voce è rappresentata dal **Process Control Block (PCB)**. Il PCB, a sua volta, è composto da due strutture dati: La **Process Structure** e la **User Structure**.



- La **process structure**, **non swappabile**, che contiene dati persistenti e gestionali fondamentali come il **PID**, lo **stato del processo**, i **puntatori a dati e stack**, il **PID del processo padre**, la **priorità**, e il **puntatore alla user structure**.
- La **user structure**, **swappabile**, che raccoglie informazioni valide solo quando il processo è residente in memoria centrale, come la **copia dei registri della CPU**, le **risorse allocate** (file aperti), la gestione degli **eventi asincroni**, la **directory corrente**, il **gruppo**, e i parametri come `argv[]`, `PATH`, ecc.

Accanto alla process table troviamo la **text table** (non swappabile), anch'essa residente e gestita dal kernel. Essa contiene i **puntatori ai segmenti di codice eseguibile condiviso**, detti **segmenti di testo**. Ogni voce della tabella è una **text structure**, che mantiene il riferimento al **codice rientrante** (eseguibile da più processi senza conflitto) e il **conteggio dei processi** che lo utilizzano.



Durante il **ciclo di vita**, un processo Unix attraversa diversi **stati operativi**. Inizia in stato **Init**, quando viene caricato in memoria e le strutture dati vengono inizializzate; passa quindi a **Ready**, quando è pronto per essere schedato; entra in **Running** quando utilizza la CPU ed è in RAM; può transitare in **Sleeping** (o **Blocked**) se è in attesa di un evento esterno; oppure in **Swapped**, quando lo scheduler a medio termine (**swapper**) ne trasferisce una parte in memoria secondaria per ottimizzare l'uso della RAM. Il processo può poi terminare regolarmente entrando nello stato **Terminated**, con la deallocazione della memoria, oppure nello stato **Zombie**, se è concluso ma ancora presente in tabella in attesa che il processo padre ne rilevi lo stato tramite funzioni come `wait()`.

Il **meccanismo di swapping**, essenziale per la gestione efficiente della memoria, prevede due operazioni: lo **swap out**, in cui vengono trasferiti in memoria secondaria i processi inattivi, in particolare quelli più lunghi; e lo **swap in**, che riporta in RAM i processi più brevi per un reinserimento rapido e poco oneroso. Questo processo è orchestrato dallo **scheduler a medio termine**,

noto come **swapper**, che seleziona i candidati in base a criteri come il tempo di attesa, la dimensione del processo e la durata della permanenza in RAM.

Alla base della gerarchia dei processi Unix si trovano alcuni **processi fondamentali**. Il primo processo utente è **init()**, identificato dal **PID 1**. Viene lanciato dal kernel al termine della fase di **bootstrap** e ha il compito di creare tutti gli altri processi del sistema, come **getty()**, responsabile della gestione del login. Inoltre, **init()** adotta eventuali **processi orfani**, evitando che restino in stato **zombie**. Nelle versioni moderne di Unix o Linux, **init()** è stato spesso sostituito da **systemd**, che conserva però le stesse funzioni di base. Il **processo con PID 0**, denominato **swapper o scheduler**, non ha un file eseguibile associato, poiché è integrato nel kernel. Accanto a esso esistono altri **processi kernel specializzati**, come **pagedaemon** (PID 2), incaricato della **paginazione della memoria virtuale**, cioè della gestione automatica del trasferimento dei dati tra RAM e memoria secondaria.

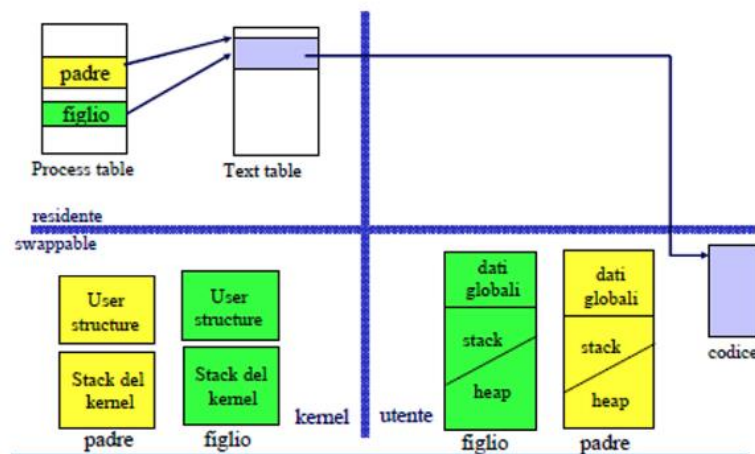
Questa architettura articolata e modulare consente a Unix di garantire un **controllo flessibile e preciso** sull'esecuzione dei processi, sull'allocazione delle risorse e sull'interazione tra componenti utente e kernel, rendendolo un sistema operativo particolarmente **adatto a scenari multiutente e multitasking**.

System call fork()

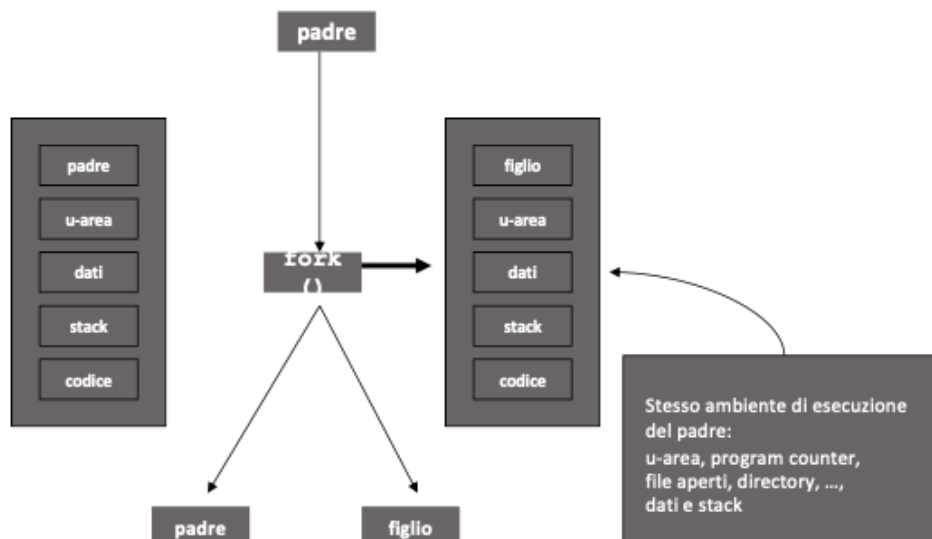
```
#include<unistd.h>

pid_t fork (void)
```

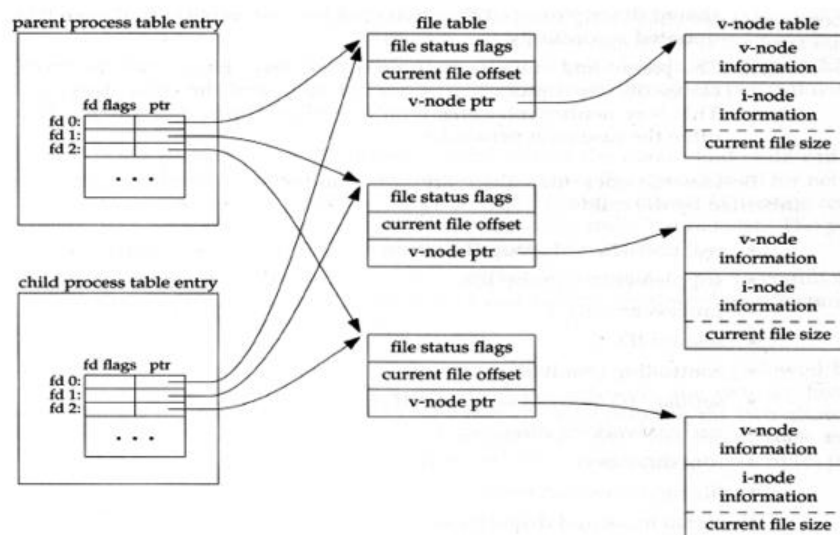
Nel sistema operativo **Unix**, la chiamata di sistema **fork()** rappresenta il meccanismo fondamentale per la **creazione di nuovi processi**. Quando un processo invoca **fork()**, il **kernel** genera una **copia quasi completa** del processo chiamante, creando un **nuovo processo figlio**. Questo processo appena creato è inizialmente **identico** al padre in termini di contenuto delle aree di memoria e contesto di esecuzione, ma possiede una propria voce indipendente nella **process table**.



Nel dettaglio, **fork()** determina la duplicazione della **process structure**, che viene inserita nella process table e inizializzata copiando le informazioni dal padre (come il PID del padre, stato, priorità, puntatori a dati e stack). Contestualmente viene allocata una **nuova user structure (u-area)** per il processo figlio, che contiene una copia dei **registri della CPU**, dei **segnali in sospeso**, delle **informazioni I/O** e di altri dati rilevanti per la gestione del processo da parte del kernel. La text structure, che rappresenta il **codice eseguibile condiviso**, non viene duplicata: essendo **rientrante** e quindi non modificabile, può essere utilizzata simultaneamente da più processi. L'unico cambiamento consiste nell'incremento del contatore nella **text table**, che registra il numero di processi che condividono quel codice.



Per quanto riguarda le **aree utente**, `fork()` comporta la **duplicazione dello stack, dell'heap e dei dati globali**, rendendo padre e figlio indipendenti nella gestione della propria **memoria privata**. Tuttavia, il **codice** resta condiviso, poiché non comporta rischi di interferenze. Anche la **directory corrente**, i segnali pendenti, il program counter e le variabili d'ambiente vengono inizialmente ereditati dal figlio.



Un aspetto rilevante della `fork` riguarda la **duplicazione dei descrittori di file**. Ogni processo possiede una **tabella locale dei descrittori**, che mappa ciascun file descriptor a una voce della **file table**, struttura condivisa che contiene lo **stato del file**, l'**offset corrente di lettura/scrittura**, e un **puntatore al vnode**, che a sua volta collega al **file system** tramite la **inode**. Con `fork()`, la tabella dei descrittori viene **copiata nel figlio**, ma i descrittori **puntano agli stessi oggetti** nella file table: questo significa che **padre e figlio condividono lo stato dei file aperti**, incluso l'offset. Qualsiasi operazione di I/O effettuata da uno sarà visibile all'altro, a meno che non si utilizzino meccanismi per duplicare o chiudere i descrittori separatamente.

La funzione `fork()` restituisce valori diversi nei due processi: il **figlio riceve 0**, mentre il **padre riceve il PID del figlio**. In caso di errore (ad esempio, per esaurimento delle risorse di sistema), `fork()` restituisce **-1** e imposta la variabile **errno**.

Un'altra caratteristica importante è che `fork()` **restituisce il controllo a due processi distinti**, e l'ordine con cui questi continueranno l'esecuzione non è deterministico. Padre e figlio iniziano dallo **stesso punto del programma**, ma possono intraprendere **percorsi diversi**, ad esempio eseguendo codice condizionato al valore di ritorno della `fork`. Quando il **processo figlio termina** la propria esecuzione, generalmente tramite la chiamata a `exit()`, la sua terminazione viene notificata al padre attraverso il **segnale SIGCHLD**. Il processo padre può quindi recuperare lo stato di uscita del figlio usando una chiamata come `wait()`, evitando che il figlio rimanga in stato di **zombie**.

In ambienti **Linux**, la **creazione di un nuovo processo** attraverso la chiamata di sistema `fork()` è resa particolarmente efficiente grazie a sofisticate **ottimizzazioni nella gestione della memoria**. In particolare, il **segmento di testo**, che contiene il

codice eseguibile, viene **condiviso** tra processo padre e processo figlio, ed è impostato in **sola lettura**, permettendo l'accesso simultaneo senza rischio di conflitti.

Per i **segmenti di memoria scrivibili**, come lo **stack**, l'**heap** e i **dati globali**, il sistema adotta una tecnica nota come **Copy-On-Write (COW)**. Con questa strategia, al momento della `fork()`, le pagine non vengono subito duplicate: padre e figlio condividono inizialmente le stesse aree fisiche. Solo quando uno dei due processi tenta di modificare una di queste aree, il **kernel** alloca una nuova pagina, copia il contenuto originale e rende la nuova pagina **privata** al processo che l'ha modificata. Questo approccio assicura a ciascun processo una **vista coerente e indipendente** della propria memoria, senza costi inutili di copia.

Grazie alla Copy-On-Write, la `fork()` diventa una chiamata **leggera e performante**, evitando la duplicazione dell'intero spazio di indirizzamento virtuale del processo padre. Le pagine vengono realmente duplicate **solo quando necessario**, il che rende `fork()` ideale anche in scenari ad **alta frequenza di creazione di processi**.

Identificativi di Processo

```
#include <unistd.h>
#include <sys/types.h>

pid_t getpid(void)    process ID
pid_t getppid(void)   parent process ID
uid_t getuid(void)    real user ID
uid_t geteuid(void)   effective user ID
gid_t getgid(void)    real group ID
gid_t getegid(void)   effective group ID
```

Nel sistema operativo Unix, ogni processo è identificato da una serie di **identificativi numerici non negativi**, che permettono di distinguere in modo univoco sia l'identità del processo stesso, sia i privilegi associati all'utente e al gruppo che lo esegue. Tali identificativi sono ottenibili attraverso specifiche **chiamate di sistema**, il cui prototipo è definito nelle intestazioni `<unistd.h>` e `<sys/types.h>`.

Tra questi:

- il **`getpid()`** restituisce il **Process ID (PID)** del processo corrente,
- **`getppid()`** fornisce il **Parent Process ID (PPID)**, ovvero il PID del processo genitore.
- **`getuid()`** restituisce il **Real User ID**, cioè l'identificativo dell'utente proprietario del processo
- **`geteuid()`** fornisce l'**Effective User ID**, che viene utilizzato per determinare i permessi effettivi durante l'accesso a file e risorse.

La stessa distinzione si applica ai **Group ID**:

- **`getgid()`** per il gruppo reale
- **`getegid()`** per il gruppo effettivo.

NB: il processo `init()`, che ha PID 1, avrà sempre PPID pari a 1 se è il capostipite.

Esempio

```
#include <stdio.h>
#include <unistd.h>
int main(void) {
    int x;
    x = 0;
    fork();
    x = 1;
    printf("process %d, x = %d\n", getpid(), x);
    return 0;
}
```

Nel primo esempio, viene mostrato che, dopo una chiamata a `fork()`, **sia il padre che il figlio continuano l'esecuzione dello stesso codice**, ma come processi distinti. Entrambi assegnano `x = 1` e stampano il loro PID con il valore aggiornato. Questo dimostra che le **variabili vengono duplicate**, ma le modifiche in uno dei due processi **non influenzano** l'altro, grazie alla separazione della memoria.

```
process 3678, x=1
process 3679, x=1
```

Esempio: creazione di una catena di n processi

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
int main (int argc, char *argv[]) {
    pid_t childpid = 0;
    int i, n;
    if (argc != 2){ /* controllo argomenti */
        fprintf(stderr, "Uso: %s processi\n", argv[0]);
        return 1;
    }
    n = atoi(argv[1]);
    for (i = 1; i < n; i++)
        if (childpid = fork())
            break;

    printf("i:%d processo ID:%d padre ID:%d figlio ID:%d\n", i, getpid(), getppid(), childpid);
    exit(0);}

```



L'esempio analizzato illustra la **creazione controllata di una catena di processi** attraverso l'uso della chiamata di sistema **fork()**, inserita all'interno di una struttura iterativa `for`. Il programma riceve come argomento da linea di comando il numero di processi da generare e, a partire da esso, costruisce una sequenza lineare in cui **ogni processo crea un solo figlio**. Questo avviene mediante un meccanismo condizionato: dopo l'invocazione di `fork()`, **solo il processo padre esce dal ciclo** utilizzando un'istruzione `break`, mentre **il processo figlio continua l'iterazione**, proseguendo nella catena.

Il risultato è una struttura gerarchica in cui **ogni processo è padre del successivo**, ma, a differenza di una ramificazione ad albero, si ottiene una **struttura lineare**, in cui ciascun nodo ha al massimo un discendente diretto.

```
alex@debianbox:~/Documenti/VsCode/Lezione_7_Processi/02.Creazione_catena_n_proce  
ssi$ ./Esempio 3  
i: 1 processo ID: 3805 padre ID: 3781 figlio ID: 3806  
i: 2 processo ID: 3806 padre ID: 3805 figlio ID: 3807  
i: 3 processo ID: 3807 padre ID: 3806 figlio ID: 0
```

Durante l'esecuzione, ogni processo stampa il proprio identificatore di processo (**PID**), il **PID del proprio padre (PPID)** e, se disponibile, il **PID del figlio appena creato**. Il figlio, ricevendo valore zero dalla `fork()`, procede nella generazione della catena, mentre il padre, ricevendo un valore positivo, esce dal ciclo ma **rimane attivo** per completare l'output previsto e terminare ordinatamente tramite la chiamata `exit()`.

Questa tecnica consente di costruire **una catena lineare di esecuzione** tra processi, evidenziando chiaramente la distinzione tra **padre e figlio**, nonché il comportamento deterministico della logica condizionale applicata alla `fork()`. L'esempio è particolarmente efficace per mostrare il funzionamento delle **relazioni di parentela** tra processi in ambiente Unix e l'utilizzo controllato delle **strutture iterative** per governare la loro proliferazione.

Altro Esempio

```
/* Stampa vari user e group ID per un processo */  
  
#include <stdio.h>  
#include <unistd.h>  
int main(void) {  
    printf("Mio real user ID: %5d\n", (uid_t)getuid());  
    printf("Mio effective user ID:%5d\n", (uid_t)geteuid());  
    printf("Mio real group ID:%5d\n", (gid_t)getgid());  
    printf("Mio effective group ID:%5d\n", (gid_t)getegid());  
    return 0;  
}
```

L'esempio fornito nel codice mostra un semplice programma che, tramite `printf()`, stampa i vari identificativi associati al processo corrente.

```
alex@debianbox:~/Documenti/VsCode/Lezione_7_Processi/03.Stampa_UserID_GroupID$ .  
/Es  
Mio real user ID: 1000  
Mio effective user ID: 1000  
Mio real group ID: 1000  
Mio effective group ID: 1000
```

L'esecuzione di questo programma consente di osservare il valore degli ID utente e di gruppo, sia reali sia effettivi, evidenziando così la distinzione tra **identità anagrafica** del processo e **identità operativa**.

Esempio: myfork.c

```
/* Un programma che si sdoppia e mostra il PID e PPID dei due processi componenti */
#include <stdio.h>
int main (int argc, char *argv[])
{
    int pid;
    printf ("Sono il processo di partenza con PID %d e PPID %d.\n",getpid(),getppid());
    pid = fork (); /* Duplicazione. Figlio e genitore continuano da qui */
    if (pid != 0) { /* pid diverso da 0, sono il padre */
        printf ("Sono il processo padre con PID %d e PPID %d.\n",getpid(),getppid());
        printf ("Il PID di mio figlio e\' %d.\n", pid); /* non aspetta con wait(); */
    }
    else { /* il pid è 0, quindi sono il figlio */
        printf ("Sono il processo figlio con PID %d e PPID %d.\n",getpid(),getppid());
    }
    printf ("PID %d termina.\n",getpid());
    /* Entrambi i processi eseguono questa parte */
    return 0;}

```

L'esempio proposto con il file **myfork.c** ha lo scopo di illustrare in modo chiaro il comportamento della chiamata di sistema **fork()**, mostrando la relazione tra processo padre e processo figlio in termini di identificativi di processo.

```
$ ./myfork
Sono il processo di partenza con PID 724 e PPID 572.
Sono il processo padre con PID 724 e PPID 572.
Sono il processo figlio con PID 725 e PPID 724.
PID 725 termina.
IL PID di mio figlio è 725
PID 724 termina.
$
```

All'avvio, il programma stampa il **PID** e il **PPID** del processo originale. In seguito, viene eseguita la **fork()**, che genera una **duplicazione del processo**. Da questo punto, entrambi i processi – padre e figlio – proseguono l'esecuzione dello stesso codice ma in rami differenti: il **padre riconosce la sua condizione** in quanto la **fork()** restituisce un valore positivo, corrispondente al PID del figlio appena creato; al contrario, il **figlio riceve zero**, identificando sé stesso come tale.

Il processo padre, dopo aver stampato il PID del figlio, **non attende la terminazione** di quest'ultimo. Prosegue infatti immediatamente e raggiunge la parte comune del codice, dove entrambi i processi stampano il proprio PID e terminano. L'assenza di un meccanismo di attesa (come **wait()**) porta a un comportamento in cui **il padre può terminare prima del figlio**. In tal caso, il sistema operativo gestisce il processo figlio come **processo orfano**, il cui genitore diventa automaticamente il **processo init()**, che ha PID 1.

Esempio: le modifiche alle variabili del processo figlio non si estendono ai valori delle variabili del processo padre

```
#include "apue.h"
int      glob=6; /* variabile esterna (blocco dati inizializzati) */
char buf[] = "una write sullo stdout\n";
int main(void)
{
    int  var; /* variabile automatica sullo stack */
    pid_t pid;
    var = 88;
    if (write(STDOUT_FILENO,buf, sizeof(buf)-1) != sizeof(buf)-1)
        err_sys("errore della write");
    printf("prima della fork\n");
    if ((pid = fork())<0) {
        err_sys("errore della fork");
    } else if (pid ==0){ /* figlio */
        glob++;          /* modifica le variabili */
        var++;
    } else {
        sleep(2);        /* padre */
    }
    printf("pid = %d, glob = %d, var = %d\n",getpid(),glob,var);
    exit(0);
}
```

Il programma dimostra che, dopo una **fork()**, padre e figlio operano su **copie indipendenti della memoria**; quindi, **le modifiche del figlio non influenzano il padre**. La funzione **write()**, non essendo bufferizzata, scrive immediatamente e compare una sola volta nell'output. Al contrario, **printf()** è **bufferizzata**: in esecuzione interattiva il buffer viene svuotato subito, ma se l'output è rediretto su file, il buffer viene **copiato in entrambi i processi**, portando a un'eventuale **duplicazione dell'output**. L'esempio evidenzia sia la **separazione dello spazio di memoria**, sia il ruolo della **bufferizzazione dell'I/O** nei processi duplicati.

```
$ ./a.out
una write sullo stdout
prima della fork
pid = 430,  glob = 7,  var = 89
    le variabili del figlio sono cambiate
pid = 429,  glob = 6,  var = 88
    la copia del padre non è cambiata
$ ./a.out > temp.out
$ cat temp.out
una write sullo stdout
prima della fork
pid = 432, glob = 7, var = 89
prima della fork
pid = 431, glob = 6, var = 88
```

Chiamata di sistema vfork()

Accanto a **fork()**, esiste una variante chiamata **vfork()**, progettata per scenari molto specifici, ovvero quando il processo figlio **esegue immediatamente una exec()** oppure termina con **_exit()** senza necessità di eseguire altro codice. A differenza

di `fork()`, `vfork()` **non duplica lo spazio di indirizzamento**: padre e figlio **condividono la stessa memoria** fino al termine del figlio o alla chiamata di `exec()`. Durante questo intervallo, il **padre è sospeso** e riprende l'esecuzione solo quando il figlio ha terminato, evitando la duplicazione di strutture dati come la tabella delle pagine.

Tuttavia, questa condivisione completa introduce **rischi significativi**: eventuali **scritture effettuate dal figlio** sono **visibili anche al padre**, portando a **effetti collaterali imprevedibili**, come modifiche non desiderate alle variabili globali. Inoltre, il figlio **non deve chiamare `exit()`**, ma deve utilizzare `_exit()`, per evitare la chiusura degli stream I/O condivisi, che potrebbe interferire con il padre.

`vfork()` fu introdotta originariamente nei sistemi **BSD** per **evitare i costi della duplicazione della memoria**, soprattutto in contesti dove il figlio invocava immediatamente `exec()`. Tuttavia, con l'adozione diffusa della **Copy-On-Write** nelle moderne implementazioni di `fork()`, il vantaggio prestazionale di `vfork()` è diventato **irrilevante**. Nei sistemi Linux contemporanei, infatti, `vfork()` è **deprecata**: non offre più benefici reali rispetto a `fork()` e presenta un **potenziale pericolo** in termini di integrità del processo padre.

Esempio: `vfork()`

Con `vfork()` il figlio **condivide lo stesso spazio di indirizzamento** del padre **fino a quando non chiama `exec()` o `_exit()`**.

Questo significa che **le variabili globali e automatiche sono condivise** e ogni modifica fatta dal figlio è visibile al padre. Questo perché il figlio non ha una copia della memoria, quindi modifica direttamente le variabili del padre fino a `_exit()`.

```
#include "apue.h"
int      glob=6; /* variabile esterna (blocco dati inizializzati) */
int main(void)
{
    int  var; /* variabile automatica sullo stack */
    pid_t pid;
    var = 88;
    printf("prima della fork\n");

    if ((pid = vfork())<0) {
        err_sys("errore della vfork");
    } else if (pid ==0){ /* figlio */
        glob++;          /* modifica le variabili */
        var++;
        _exit(0);        /* il figlio finisce */
    }
    /* il padre continua qui */
    printf("pid = %d, glob = %d, var = %d\n",getpid(),glob,var);
    exit(0);
}
```

```
$ ./a.out
```

```
Prima della vfork
```

```
Pid = 29039, glob = 7, var = 89
```

- L'incremento delle variabili fatte dal figlio modifica i valori nel genitore

Terminazione di processi in Unix

Nel sistema **Unix**, la **terminazione di un processo** implica una sequenza di operazioni volte a garantire che lo stato del sistema rimanga coerente. Questo processo prevede la **chiusura automatica dei file aperti**, la **rimozione dell'immagine del processo dalla memoria** e, se necessario, l'invio di una **segnalazione al processo padre** per informarlo dell'avvenuta conclusione.

Per gestire correttamente la terminazione, Unix coordina l'uso della funzione **exit**, che consente al processo di concludersi, con le chiamate di sistema **wait** o **waitpid**, che permettono al processo padre di **attendere la terminazione del figlio**. Questa interazione è essenziale per garantire la corretta **deallocazione delle risorse** e per impedire che i processi terminati rimangano nello stato di **zombie**, occupando inutilmente risorse del sistema.

Chiamate di sistema wait() e waitpid()

```
#include <sys/types.h>
#include <sys/wait.h>

pid_t wait(int *statloc)
pid_t waitpid(pid_t pid, int *statloc, int options);
```

Nel sistema **Unix**, la **terminazione di un processo** è un evento gestito con attenzione dal sistema operativo per garantire la **coerenza** e il corretto **rilascio delle risorse**. Quando un processo termina, il **kernel** provvede automaticamente alla **chiusura dei file aperti**, alla **rimozione dell'immagine del processo dalla memoria**, e all'**invio di una notifica** al processo padre, informandolo della conclusione del figlio tramite il **segnale SIGCHLD**.

Per permettere al processo padre di **gestire la terminazione dei figli**, Unix mette a disposizione due chiamate di sistema: **wait()** e **waitpid()**. Entrambe le funzioni consentono al padre di **attendere** che uno o più processi figli terminino, e di **recuperare il loro stato di uscita**.

La funzione **wait()** blocca il processo chiamante finché **uno qualsiasi** dei suoi figli non termina o finché **non riceve un segnale**. Restituisce il **PID del processo figlio terminato** oppure **-1** in caso di errore (ad esempio, se il processo non ha figli). Se il figlio è già in stato **zombie** (cioè è terminato ma non ancora gestito), la funzione **restituisce immediatamente**.

La funzione **waitpid()** offre **maggiore flessibilità**, permettendo di **attendere un figlio specifico** o un gruppo di figli, in base al valore dell'argomento **pid**. Anche **waitpid()** blocca il chiamante di default, ma può essere configurata per **non bloccare**, tramite l'uso dell'opzione **WNOHANG**. I significati principali dell'argomento **pid** sono:

- **pid == -1**: attende **qualsiasi** processo figlio (comportamento equivalente a **wait()**),
- **pid > 0**: attende **uno specifico** processo figlio con quel **PID**,
- **pid == 0**: attende un figlio il cui **Group ID** è uguale a quello del chiamante,
- **pid < -1**: attende un figlio il cui **Group ID** è uguale al **valore assoluto** dell'argomento **pid**.

Entrambe le funzioni accettano un argomento **statloc**, che è un **puntatore a intero**. Se non è **NULL**, questo puntatore viene usato per **memorizzare lo stato di terminazione** del figlio. Questo stato è **codificato in un intero**, in cui il **byte meno significativo** indica la causa della terminazione (ad esempio un segnale), mentre il **byte più significativo** rappresenta il codice di uscita restituito dal processo figlio con la funzione **exit()**.

Per interpretare correttamente questo valore, si usano le **macro definite in <sys/wait.h>**, che operano sul valore intero e **non sul puntatore**. Le principali sono:

- **WIFEXITED(status)**: restituisce vero se il figlio è terminato normalmente con **exit()**,
- **WEXITSTATUS(status)**: restituisce il codice di uscita fornito da **exit()**,
- **WIFSIGNALED(status)**: vero se il figlio è terminato a causa di un **segnale**,
- **WTERMSIG(status)**: restituisce il **numero del segnale** che ha causato la terminazione,
- **WIFSTOPPED(status)**: vero se il figlio è stato **sospeso** (es. da **SIGSTOP**),
- **WSTOPSIG(status)**: restituisce il **segnale che ha causato la sospensione**.

L'argomento **options** di **waitpid()** permette di personalizzare ulteriormente il comportamento della chiamata. Le opzioni principali sono:

- **WNOHANG**: fa sì che la funzione **non blocchi** il padre se non vi sono figli già terminati; in tal caso, restituisce 0. Questa opzione è utile in **cicli di polling** o applicazioni non bloccanti.
- **WUNTRACED**: permette di ottenere informazioni anche sui **figli sospesi**, non solo su quelli terminati.

Chiamata exit()

```
void exit(int status);
```

La chiamata di sistema **exit(int status)** ha il compito di **terminare il processo chiamante** e **rendere disponibile un valore di uscita**, chiamato **status**, al **processo padre**. Questo valore viene poi recuperato dal padre attraverso le funzioni **wait** o **waitpid**.

Nel caso di una **terminazione normale**, lo stato di uscita coincide con il valore passato alla funzione `exit()` oppure con il valore di ritorno della funzione `main`. Se invece il processo viene **terminato in modo anomalo** (per esempio tramite un segnale), non sarà `exit()` a fornire uno `status`, ma sarà il **kernel** stesso a generare un **termination status** che indica il motivo della terminazione.

È importante distinguere tra **exit status** e **termination status**: il primo è esplicitamente specificato dal programma nel caso di conclusione corretta, mentre il secondo viene prodotto automaticamente dal sistema in caso di errori o interruzioni forzate. Entrambi i valori sono resi disponibili al padre, ma attraverso meccanismi diversi.

```
#include "apue.h"
#include <sys/wait.h>
void pr_exit (int status)
{
    if (WIFEXITED(status))
        printf("term. normale, exit status =%d\n", WEXITSTATUS(status));
    else if (WIFSIGNALED(status))
        printf("term. anomala", num. segnale =%d\n", WTERMSIG(status));
    else if (WIFSTOPPED(status))
        printf("figlio fermato, num. segnale %d\n", WSTOPSIG(status));
}
```

L'esempio mostrato nel codice usa la funzione `pr_exit(int status)` per interpretare lo stato del processo figlio al termine della sua esecuzione. Viene utilizzato il valore intero `status` ritornato da `wait()` o `waitpid()` e analizzato tramite **macro** definite in `<sys/wait.h>`:

- `WIFEXITED(status)` verifica se il processo è terminato normalmente. In tal caso, `WEXITSTATUS(status)` restituisce il valore di uscita.
- `WIFSIGNALED(status)` controlla se il processo è stato interrotto da un segnale e `WTERMSIG(status)` indica quale segnale l'ha causato.
- `WIFSTOPPED(status)` indica se il processo è stato sospeso da un segnale e `WSTOPSIG(status)` ne mostra il codice.

Processi zombie

Quando un **processo** termina, non viene immediatamente rimosso dal sistema. Rimane invece presente fino a quando il **processo padre** non ne raccoglie il **codice di terminazione**. In questa fase intermedia, il processo viene definito **zombie**. Un processo zombie è riconoscibile nel comando `ps` tramite l'etichetta `<defunct>`. Finché il padre non chiama esplicitamente una funzione come **wait()**, il codice di uscita del processo figlio non viene acquisito, e il figlio rimane nello stato di zombie.

Sebbene un **processo zombie** non utilizzi più risorse come **codice**, **dati** o **pila**, esso conserva un **PCB** (Process Control Block), una struttura presente nella **Process Table**, la cui dimensione è limitata. Un numero eccessivo di zombie può quindi saturare questa tabella.

Processi adottati

Nel caso in cui un processo padre termini prima del proprio figlio, quest'ultimo è detto **orfano**. Il **kernel** garantisce che tutti i processi orfani vengano **adottati** da **init()**, a cui viene assegnato come **PPID** il valore 1. Quando un processo adottato da **init** termina, non diventa zombie. Infatti, **init** è progettato in modo da chiamare sempre una delle funzioni **wait** per recuperare lo **stato di uscita** di tutti i processi figli, inclusi quelli adottati. Ciò permette di prevenire l'accumulo di zombie nel sistema.

Race Conditions

Un concetto critico è rappresentato dalle **race condition**, che si verificano quando più processi accedono a **dati condivisi**. In tali situazioni, il risultato può dipendere dall'**ordine di esecuzione** dei processi, che non è prevedibile a priori, essendo determinato dalla **politica di scheduling** del **kernel** e dal carico complessivo del sistema. Per evitare comportamenti non deterministici, è necessario implementare **meccanismi di sincronizzazione** tra i processi, o utilizzare strumenti di **comunicazione interprocesso (IPC)**.

Famiglia exec

Nel sistema **Unix**, la **famiglia di funzioni exec** svolge un ruolo fondamentale nella **programmazione dei processi**, consentendo a un processo esistente di **sostituire completamente la propria immagine di esecuzione** con quella di un **nuovo programma**. Questo meccanismo è particolarmente utile nei casi in cui un processo viene creato, ad esempio con **fork()**, al solo scopo di eseguire un **programma diverso**, come accade comunemente nelle **shell** o nei **server**, dove il figlio gestisce connessioni o attività separate.

Quando una funzione **exec** viene invocata, il **codice**, i **dati**, lo **stack** e l'**heap** del processo corrente vengono **completamente rimpiazzati** con quelli del nuovo programma caricato dal disco. La precedente immagine di memoria **viene cancellata definitivamente**, rendendo impossibile un ritorno al programma originario. Tuttavia, alcune componenti del processo **restano invariate**: la **process structure** (inclusi PID e relazioni gerarchiche), la **user area** (eccetto le informazioni legate al codice e al program counter), e lo **stack del kernel**. Anche le **risorse aperte**, come file descriptor e variabili d'ambiente, **possono essere mantenute**, a meno che il nuovo programma non le modifichi esplicitamente.

Dal punto di vista del sistema operativo, il processo **non cambia identità**: mantiene il proprio **PID**, continua a essere gestito dalla stessa voce nella **process table**, e può essere **atteso dal padre** tramite le funzioni **wait()** o **waitpid()**, esattamente come avverrebbe se il figlio avesse eseguito codice C senza chiamare **exec**.

Le varianti della famiglia exec

Le funzioni della famiglia **exec** si distinguono per tre aspetti principali:

1. **Modalità di passaggio degli argomenti**: tramite **lista (1)** o **array (v)**.
2. **Gestione dell'ambiente**: se la funzione ha il suffisso **e**, consente di specificare esplicitamente un **nuovo ambiente envp[]**.
3. **Metodo di ricerca del file eseguibile**: con il suffisso **p**, il file viene cercato nel **PATH** della shell; in sua assenza, serve un **percorso assoluto**.

Funzioni con lista di argomenti:

```
int execl (const char *path, const char *arg0, ..., (char *) 0);
int execl_e (const char *path, const char *arg0, ..., (char *) 0,
char *const *envp[]);
int execlp (const char *file, const char *arg0, ..., (char *) 0);
```

- **execl**: accetta un elenco variabile di argomenti.

- **execle**: come `execl`, ma consente di passare anche un ambiente personalizzato.
- **execlep**: come `execl`, ma cerca il file eseguibile nel `PATH`.

Funzioni con array di argomenti (vector):

```
int execv (const char *path, char *const argv[]);
int execve (const char *path, char *const argv[], char *const envp[]);
int execvp (const char *file, char *const argv[]);
```

- **execv**: accetta un array `argv[]` terminato da `NULL`.
- **execve**: come `execv`, ma consente anche un ambiente personalizzato.
- **execvp**: come `execv`, ma utilizza il `PATH` per trovare l'eseguibile.

Funzione centrale: `execve()`

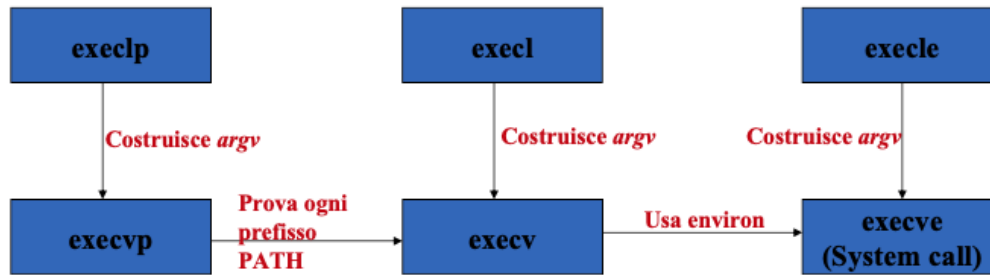
Tra tutte, **`execve()`** è l'unica **vera system call** gestita direttamente dal kernel. Le altre funzioni (`execl`, `execv`, ecc.) sono semplici **wrapper di libreria**, che costruiscono i parametri corretti per poi invocare internamente `execve()`. Essa riceve tre argomenti:

```
int execve (char *filename, char *argv[], char *envp[]);
```

- **filename**: percorso completo del file eseguibile,
- **argv[]**: array degli argomenti, terminato da `NULL`,
- **envp[]**: array dell'ambiente, anch'esso terminato da `NULL`.

Tutte le funzioni della famiglia **exec** restituiscono **-1** in caso di errore; in caso di successo, **non ritornano** mai, poiché l'immagine del processo è stata sostituita.

Relazione tra le funzioni della famiglia exec



La relazione tra queste funzioni può essere riassunta come segue:

- `execl`, `execle` e `execip` **costruiscono `argv`** e poi invocano rispettivamente `execv`, `execve`, o `execvp`;
- `execvp` si occupa anche di **provare i prefissi definiti nel `PATH`** per trovare l'eseguibile;
- `execv` utilizza direttamente `execve` passando l'ambiente corrente (`environ`);
- `execve` è l'unica a interagire direttamente con il **kernel**, ed è quella che effettivamente sostituisce l'immagine del processo.

Schema di chiamate a fork, exec e wait (waitpid)

```
esito = fork(); /* crea un processo figlio */
if (esito < 0) /* creazione OK? */
{ error("fork() non eseguita");
...
}
if (esito == 0) /* è il processo figlio ? */
{exec("p",...); /* si esegue il programma "p" */
...           /* ...a meno di errori */
}
id = wait(&stato); /* il processo padre attende la
                  fine */
if (stato == ...) /* del figlio e ne analizza
                  l'esito */
{
...
}
```

Lo schema rappresenta il comportamento classico di un processo che utilizza le chiamate di sistema **fork()**, **exec()** e **wait()** (o **waitpid()**) per creare un nuovo processo, eseguire un programma e attendere la sua conclusione. Il processo iniziale chiama **fork()**, che serve a duplicare il processo corrente. A partire da questo momento esistono due processi distinti: il **padre** e il **figlio**, entrambi eseguono lo stesso codice ma con valori diversi di ritorno da **fork()**. Se il valore restituito è **minore di zero**, significa che la creazione del figlio è fallita e viene generato un **errore**. Se invece il valore è **uguale a zero**, ci troviamo nel **processo figlio**.

Il figlio, una volta creato, prosegue l'esecuzione e invoca una chiamata della famiglia **exec** per sostituire la propria immagine in memoria con quella di un nuovo programma, ad esempio "p". Questo implica che lo **stack**, lo **heap**, i **dati** e il **testo** del processo corrente vengono rimpiazzati. Se **exec()** ha successo, il nuovo programma prende il controllo e il codice successivo non viene eseguito. Se invece si verifica un errore, il controllo torna al processo figlio e sarà necessario gestirlo opportunamente.

Nel frattempo, il **processo padre** continua l'esecuzione e chiama la funzione **wait()**, che sospende l'esecuzione fino a quando il figlio non termina. Quando il figlio termina, il padre riceve il suo **PID** e può accedere al valore di **stato** che descrive la modalità con cui il figlio si è concluso. Questo valore può essere interpretato con macro apposite per determinare se la terminazione è stata **normale**, **anomala** o causata da un **segnale**.

Esempio: myexec.c

```
#include <stdio.h>

int main (void) {
    printf("Sono il processo %d, eseguo una ls -l\n",getpid());
    execl("/bin/ls", "ls", "-l", NULL); /* Esegue ls -l */
    printf("Questa riga non dovrebbe essere eseguita\n");
}
```

```
$ myexec
Sono il processo 797 e sto per eseguire una ls -l
total 3324
drwxr-xr-x 3 lagrene bireli 4096 May 16 19:14 Glass
-rwxr-xr-x 1 lagrene bireli 22199 Mar 17 18:34 lez1.sxi
-rwxr-xr-x 1 lagrene bireli 25999 May 21 17:14 lez20.sxi
$
```

Il programma **myexec.c** ha lo scopo di mostrare come funziona la chiamata alla funzione **execl()**, appartenente alla famiglia **exec**. Questo programma stampa un messaggio che indica l'identificativo del processo, poi esegue il comando **ls -l** tramite **execl()**, sovrascrivendo così la propria immagine di processo con quella del programma **ls**. È importante notare che in questo esempio non viene utilizzata alcuna chiamata a **fork()**, quindi non viene creato un nuovo processo figlio.

Quando **execl()** viene invocata con successo, il processo in esecuzione viene completamente sostituito dal programma specificato. Di conseguenza, il controllo non ritorna mai al chiamante: la riga di codice **printf("Questa riga non dovrebbe essere eseguita\n");** non verrà mai eseguita. Questo comportamento è ben illustrato nell'esecuzione mostrata: il messaggio iniziale viene stampato, seguito dalla normale uscita del comando **ls -l**, che elenca i file nella directory corrente.

Esempio: myexec1.c

```
int main(){
    int pid, status;
    pid=fork();
    if (pid==0)
    {execl("/bin/ls", "ls","-l","pippo",(char *)0);
    printf("exec fallita!\n");
    exit(1);
    }
    else if (pid >0)
    {pid=wait(&status);
    /* gestione dello stato.. */
    exit(0);
    }
    else printf("fork fallita!");
    exit(2);
}
```

Il programma **myexec1.c** illustra un tipico utilizzo combinato delle chiamate di sistema **fork()**, **execl()** e **wait()**. Il processo principale crea un processo figlio tramite **fork()**. Se la creazione ha successo e il codice si sta eseguendo nel **figlio**, questo invoca **execl()** per eseguire il comando **ls -l pippo**. Tuttavia, poiché il file "pippo" probabilmente non esiste come argomento valido per il comando **ls**, l'esecuzione fallisce. In tal caso, viene stampato il messaggio **"exec fallita!"** e il figlio termina con **exit(1)**.

Se invece il codice è in esecuzione nel **padre**, questo attende la terminazione del figlio attraverso **wait()**, ottenendo il valore di **status**, che può essere poi usato per analizzare l'esito del figlio. Dopo la gestione dello stato, il padre termina con **exit(0)**.

Infine, se **fork()** fallisce, il programma stampa un messaggio di errore e termina con **exit(2)**. L'esempio mostra chiaramente il flusso di controllo tra padre e figlio, e sottolinea come **execl()**, se va a buon fine, non ritorna mai al chiamante.

Esecuzione dei comandi nella shell

Durante l'**esecuzione dei comandi nella shell**, ogni comando digitato viene interpretato come una sequenza di stringhe separate da spazi. La prima stringa rappresenta il **comando** da eseguire, mentre le successive rappresentano gli **argomenti** da passare. Quando un comando viene lanciato, la shell esegue internamente una chiamata a **fork()**, che crea un nuovo processo figlio, e in questo figlio viene eseguita una funzione della famiglia **exec()**, che carica in memoria l'immagine del nuovo programma, sostituendo quella precedente.

Ogni comando, una volta terminato, restituisce un **exit status**, un valore intero che indica l'esito dell'esecuzione: **0** rappresenta un successo, un numero **positivo** segnala un fallimento, mentre **128 + n** indica che il processo è stato terminato da un **segnale** numero **n**. Questo valore è fondamentale per determinare se l'operazione si è svolta correttamente o meno.

Durante l'esecuzione, la **shell sospende temporaneamente** la sua operatività fino alla terminazione del comando lanciato. Solo dopo che il processo figlio termina, e la shell ha ottenuto l'exit status tramite **wait()**, essa riprende il controllo ed è pronta per accettare nuovi comandi.

Esempio: esecuzione di un comando

```
#include <stdio.h>

int main (int argc, char *argv[]) {
    if (fork () == 0) { /* Figlio */
        execvp (argv[1], &argv[1]); /* Esegue un altro programma */
        fprintf (stderr, "Non ho potuto eseguire %s\n", argv[1]);
    }
}
```

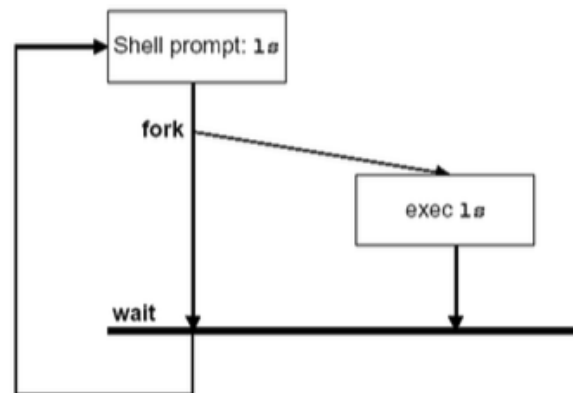
Il programma **execute cmd args** utilizza la chiamata di sistema **fork()** per creare un nuovo processo, seguito dalla chiamata **execvp()** per eseguire un comando specificato come parametro da riga di comando, insieme ai suoi argomenti.

In particolare, il processo figlio, identificato dalla condizione `fork() == 0`, chiama **execvp()**, passando come primo argomento il comando da eseguire (`argv[1]`) e come secondo argomento la lista degli argomenti del programma (`&argv[1]`). Questo passaggio consente di avviare un nuovo programma, sostituendo completamente il codice del processo figlio con quello dell'eseguibile specificato. Se il comando non può essere eseguito, viene stampato un messaggio di errore su `stderr`.

È importante ricordare che **execvp()** sfrutta la variabile di ambiente **PATH** per cercare l'eseguibile da lanciare, e che la lista di argomenti deve essere terminata da un **puntatore NULL**. Nel programma, questo è garantito dal fatto che `argv[argc] == NULL`.

```
$ execute sleep 5
$ ps
PID TTY TIME CMD
822 pts/0 00:00:00 bash
925 pts/0 00:00:01 vi
965 pts/0 00:00:00 sleep
969 pts/0 00:00:00 ps
$
```

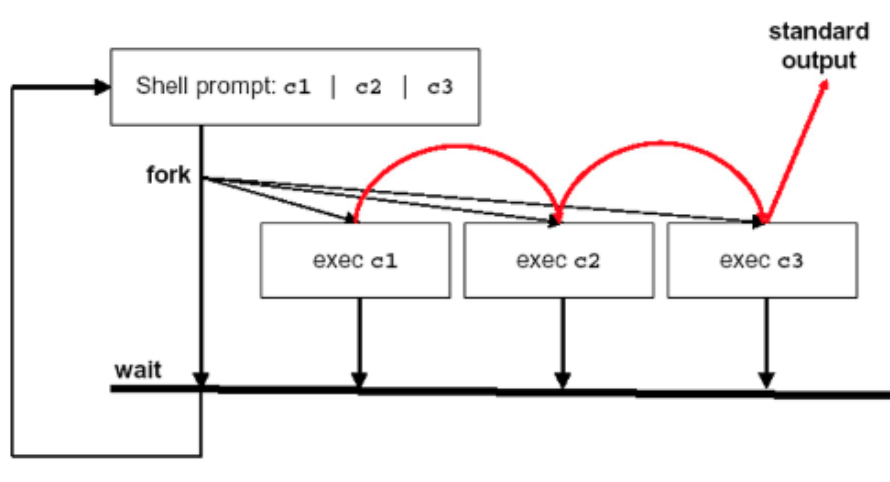
Nell'esempio mostrato a schermo, l'esecuzione del comando `exec sleep 5` crea un processo figlio che esegue il comando `sleep` per 5 secondi. Durante questo tempo, il comando `ps` mostra tra i processi attivi quello associato a `sleep`, confermando l'effettiva sostituzione dell'immagine del processo figlio con il nuovo programma.



L'intero meccanismo di esecuzione, come rappresentato nei diagrammi, mostra che la **fork** crea un nuovo processo, che eredita l'ambiente della shell, e successivamente tramite **exec**, questo processo viene trasformato nel comando richiesto (ad esempio `1s`). Questo nuovo processo condivide parte delle risorse iniziali con il genitore (in particolare il descrittore dei file aperti) ma esegue un **programma differente**, sostituendo completamente la vecchia immagine di esecuzione.

Esecuzione di una pipeline da shell

Una **pipeline** in una **shell Unix** è una sequenza di comandi collegati tra loro tramite l'operatore **pipe** (`|`). Questo operatore permette di **reindirizzare lo standard output** del primo comando nello **standard input** del comando successivo, creando così un **flusso continuo di dati** tra i diversi comandi coinvolti. Ad esempio, in un'istruzione come `comando1 | comando2 | comando3`, l'output generato da `comando1` viene automaticamente fornito come input a `comando2`, e così via fino all'ultimo comando.



Dal punto di vista dell'implementazione, per ciascun comando nella pipeline, la shell esegue una chiamata a **fork()**, creando un nuovo **processo figlio**. Ogni processo figlio, a sua volta, invoca una delle funzioni della **famiglia exec** per sostituire la propria immagine di processo con quella del comando da eseguire (ad esempio `exec comando1`, `exec comando2`, ecc.). Durante questa fase, la shell crea anche le opportune **pipe** (tramite la system call `pipe()`) per **collegare tra loro gli stream di input e output** dei processi figli, in modo da realizzare la comunicazione tra i comandi.

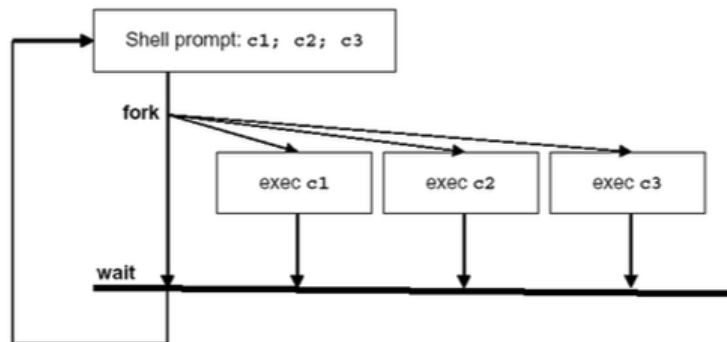
Infine, il processo padre, cioè la shell, **attende la terminazione** dei processi figli tramite una chiamata a **wait()** o **waitpid()**, garantendo che l'intera pipeline venga completata prima che il prompt della shell ritorni disponibile. Questo meccanismo permette di eseguire comandi complessi in modo fluido ed efficiente, sfruttando appieno il modello di **comunicazione tra processi** tipico dei sistemi Unix.

Liste di comandi

Una **lista di comandi** consente di eseguire in maniera **sequenziale** più comandi come se costituissero un **unico comando**. La sintassi prevede che i comandi siano separati da un **punto e virgola (;)**, come in `comando1 ; comando2 ; comando3`. Ciascun comando, compresi quelli all'interno della lista, può a sua volta essere una **pipeline**.

L'**exit status** della lista è determinato esclusivamente dall'**ultimo comando** eseguito, e rappresenta quindi l'esito complessivo della lista. La **shell** attende che **tutti i comandi** siano stati completati prima di restituire il **prompt all'utente**.

A differenza delle **pipeline**, in cui lo **standard output** di un comando viene inviato allo **standard input** del successivo, le liste di comandi **non prevedono alcun collegamento diretto** tra input e output dei comandi contenuti.



Come mostrato nello schema, la shell esegue una **fork** per ogni comando, dopodiché ogni processo figlio **invoca una exec** per sostituire la propria immagine di processo con quella del comando da eseguire. Al termine dell'esecuzione, la shell **attende (wait)** la conclusione di tutti i processi prima di tornare operativa.

Esempio: modello semplificato di shell

```
while (TRUE) {
    read_command(command, parameters);
    if (fork() != 0) {
        waitpid(-1, &status, 0);
    } else {
        execve(command, parameters, env);
    }
}
```

Questo frammento rappresenta un **modello semplificato** del funzionamento di una **shell**, ovvero di un programma che consente di interpretare ed eseguire comandi da parte dell'utente.

All'interno di un **ciclo infinito**, viene invocata la funzione `read_command`, che legge da input un **comando** e i suoi **parametri**. Successivamente, il processo invoca una **fork**: se il valore di ritorno è diverso da zero, significa che ci si trova nel **processo padre**, il quale chiama `waitpid` per **attendere la terminazione** del processo figlio e raccoglierne lo **status** di uscita. Se invece il valore ritornato è **zero**, si è nel **processo figlio**, che esegue `execve`, andando a **sostituire l'immagine di processo** con quella del comando specificato.

In questo modo, ogni comando digitato nella shell viene eseguito in un **nuovo processo**, garantendo l'indipendenza tra i vari comandi e permettendo alla shell di **restare attiva** per continuare ad accettare input.

L'esempio riportato `cp file1 file2` mostra un tipico comando con **parametri**, dove il meccanismo di gestione è del tutto analogo a quello utilizzato nella funzione `main()` di un programma in C, con i parametri accessibili tramite `argv[]`.

Esempio: creazione e attesa processi

```
...
int main (int argc, char *argv[]) {
    pid_t childpid;
    int i, n;
    pid_t waitreturn;
    if (argc != 2){ /* controllo argomenti */
        fprintf(stderr, "Uso: %s processi\n", argv[0]);
        return 1;
    }
    n = atoi(argv[1]);
    for (i = 1; i < n; i++)
        if (childpid = fork())
            break;
    while(childpid != (waitreturn = wait(NULL)))
        if ((waitreturn == -1)
            break;
    fprintf(stderr, "processo %d, padre %d\n", getpid(), getppid());
    return 0;}
```

Questo programma in C ha lo scopo di creare una **catena di processi**, dove ogni processo genera **al massimo un figlio**, e ognuno aspetta che il proprio figlio finisca prima di procedere. Si utilizza la funzione **fork()** per creare un nuovo processo e la funzione **wait()** per attendere che un figlio termini l'esecuzione.

Il programma prende come parametro il numero totale di **processi da creare**, indicato dall'utente al momento dell'esecuzione. Dopo aver convertito questo valore in intero, entra in un ciclo **for** che si ripete finché non si è raggiunto il numero desiderato di processi. Dentro questo ciclo viene chiamata `fork()`, che crea un nuovo processo. La particolarità è che **solo il processo figlio** continua il ciclo, mentre il **padre esce dal for** grazie alla condizione `if ((childpid = fork())) break;`. Questo fa sì che venga creata una **catena lineare**: ogni processo genera un solo figlio e poi esce dal ciclo.

Attenzione: quando si dice che "*il padre si ferma*", non significa che si blocca completamente o che termina l'esecuzione. Significa semplicemente che **non crea altri figli**, cioè **esce dal ciclo for**. Dopo il `for`, il padre **continua l'esecuzione del codice** e entra nel ciclo **while**, dove si **mette in attesa che il figlio appena creato termini**. Questo avviene usando la funzione `wait()`, che sospende l'esecuzione del padre finché uno dei suoi figli non termina.

Una volta che il figlio termina, il padre esce dal `while` e **stampa il proprio PID** (identificativo di processo) e quello del proprio padre (PPID). Questo comportamento si ripete per tutti i processi nella catena: l'ultimo figlio (quello che non ha creato altri) è il primo a stampare, poi il padre, poi il nonno, e così via, fino ad arrivare al processo iniziale. Il risultato è che i processi **stampano in ordine inverso alla loro creazione**, cioè **dal più giovane al più anziano**.

Grazie a questo meccanismo, il programma permette di osservare in modo ordinato e sequenziale la **creazione e sincronizzazione dei processi**, e fornisce un esempio molto chiaro del funzionamento di `fork()` e `wait()`.

Capitolo 8 – Segnali

Introduzione

I **segnali** sono meccanismi di **notifica asincrona** inviati ai processi per segnalare eventi. Sono simili alle **interruzioni hardware**, poiché possono interrompere l'esecuzione normale di un programma in momenti **imprevedibili**. Un processo può inviare segnali ad altri processi o a sé stesso, rendendoli una forma **primitiva di comunicazione interprocesso (IPC)**.

La principale fonte di segnali è il **kernel**, che li genera in risposta a **eventi hardware o software**, come **eccezioni** (divisioni per zero, accessi illegali alla memoria), input dell'utente tramite **Control-C (SIGINT)** o **Control-Z (SIGSTOP)**, **scadenze di timer**, **chiusure di finestre**, o la **terminazione di processi figli**.

Ogni segnale è rappresentato da un **intero univoco** e un **nome simbolico** (es. **SIGTERM**, **SIGSEGV**) definiti in `<signal.h>`. È cruciale usare i **nomi simbolici**, perché i numeri possono variare tra sistemi.

I segnali si dividono in **standard**, usati per notificare eventi comuni, e **real-time**, introdotti per gestioni più precise e complesse.

La **disposizione** di un segnale indica al kernel come un processo deve **reagire** alla sua ricezione. Un segnale si dice **generato** quando l'evento che lo causa avviene, e **consegnato** quando il processo esegue l'azione corrispondente. Nel frattempo, può rimanere **pendente**. Se generato più volte, il comportamento può variare a seconda del sistema.

Un processo può decidere di:

- **ignorare** un segnale;
- **intercettarlo** con una funzione specifica;
- lasciare che venga eseguita l'**azione di default**.

Tuttavia, segnali come **SIGKILL** e **SIGSTOP** non possono essere ignorati, e ignorare segnali gravi come **SIGFPE**, **SIGILL** e **SIGSEGV** può portare a comportamenti **imprevedibili**.

In genere, l'**azione di default** è la **terminazione del processo** con possibile **generazione di un core dump**, utile per la diagnosi.

Il comando `kill -l` permette di vedere l'elenco dei segnali, con nomi, numeri e funzioni. Una corretta gestione dei segnali è **fondamentale per la stabilità e affidabilità** dei sistemi, oltre a rappresentare una forma efficace di **comunicazione asincrona tra processi**.

Nel sistema **Unix**, i segnali hanno **reazioni specifiche**. Il **SIGFPE** indica errori aritmetici (es. divisione per zero) e termina il processo con core dump. Il **SIGHUP** viene inviato alla chiusura del terminale e termina i processi in background. Il **SIGINT**, generato da **CTRL-C**, termina il processo salvo gestione personalizzata. **SIGKILL**, non ignorabile o intercettabile, **termina immediatamente** il processo. **SIGSEGV** indica accessi a memoria non permessi e porta alla terminazione con core dump. **SIGUSR1** e **SIGUSR2** sono disponibili per **uso personalizzato** da parte dei programmatori.

I **segnali**, quindi, sono strumenti cruciali per la **gestione degli errori**, il **controllo dei processi** e la **comunicazione asincrona** nei sistemi operativi come Unix.

La funzione signal()

La **funzione `signal()`** è uno strumento della **libreria C** che permette a un processo di **gestire i segnali asincroni**, istruendo il **kernel** su quale **azione eseguire** al momento della loro ricezione. Questa funzione consente di associare a un segnale tre possibili comportamenti: **ignorarlo** (tramite la macro `SIG_IGN`), **eseguire l'azione di default** (`SIG_DFL`), oppure specificare una **funzione definita dall'utente** da invocare in risposta al segnale.

Il **prototipo classico** della funzione è:

```
#include <signal.h>

void (*signal(int signo, void (*func)(int)))(int);
```

Questa dichiarazione indica che `signal()` riceve due argomenti:

- **signo**, un intero che rappresenta il **numero del segnale**, tipicamente specificato tramite una **macro** definita in `<signal.h>` (es. `SIGINT`, `SIGTERM`);
- **func**, un **puntatore a funzione** che accetta un intero (il segnale ricevuto) e non restituisce nulla. Questo puntatore può essere:
 - `SIG_IGN`, per **ignorare** il segnale;
 - `SIG_DFL`, per eseguire l'**azione predefinita** del sistema;
 - l'indirizzo di una funzione **personalizzata** che definisce il comportamento desiderato.

La funzione restituisce il **puntatore alla funzione precedentemente associata** a quel segnale, utile se si desidera **ripristinare** una vecchia gestione o **verificare** l'attuale disposizione. In caso di errore, ad esempio se il segnale specificato **non esiste** o non può essere modificato, restituisce la macro `SIG_ERR`.

Per semplificare la leggibilità del prototipo, si usa spesso il **typedef**:

```
typedef void (*sighandler_t)(int);
```

Che consente di riscrivere il prototipo come:

```
sighandler_t signal(int, sighandler_t);
```

```
#define SIG_ERR          (void (*) (int) ) -1
#define SIG_DFL          (void (*) (int) )  0
#define SIG_IGN          (void (*) (int) )  1
```

Nell'header `<signal.h>` sono quindi definite anche le costanti `SIG_ERR`, `SIG_DFL` e `SIG_IGN`, che rappresentano i **comportamenti speciali associabili** a un segnale. In sintesi, `signal()` è una funzione fondamentale per la **personalizzazione del comportamento di un processo** in risposta ai segnali nel sistema operativo.

Un esempio pratico:

```
#include <signal.h>

int main(void)
{
    signal (SIGINT, SIG_IGN);
    while (1);
}
```

Nel codice sopra, il processo ignora il segnale `SIGINT` (CTRL-C), rendendo impossibile la sua interruzione tramite tastiera. Per terminare il processo sarà necessario utilizzare il comando `kill` da un'altra shell.

La funzione `signal()` rappresenta dunque un meccanismo **fondamentale di controllo** per la **gestione personalizzata dei segnali** in ambiente Unix/Linux.

Gestore dei segnali

Un **gestore di segnali** (signal handler) è una funzione che il **kernel** può invocare in qualsiasi momento per conto del processo, interrompendo il flusso normale di esecuzione del programma. Una volta completata l'esecuzione del gestore, il controllo ritorna al punto esatto in cui era stato interrotto il programma.

Anche se tecnicamente un gestore può eseguire qualsiasi operazione, per motivi di **sicurezza e affidabilità**, è consigliabile che sia **il più semplice possibile**, per evitare **race condition** e problemi di concorrenza. Una strategia comune è quella in cui il gestore si limita a modificare una **variabile globale** (un flag), che sarà poi verificata ciclicamente dal programma principale per intraprendere eventuali azioni. Un altro uso comune è quello di eseguire **operazioni di pulizia** (come la chiusura di file o la liberazione di risorse) prima della terminazione di un processo.

Quando un processo viene creato tramite la chiamata di sistema **fork()**, il processo figlio eredita **tutte le disposizioni dei segnali** del processo padre, inclusi gli **handler personalizzati** definiti tramite **signal()**. Questo avviene perché **fork()** genera una **copia identica del processo genitore**, comprese le strutture dati interne usate per la gestione dei segnali.

Tuttavia, quando un processo chiama una funzione della famiglia **exec()** per caricare un nuovo eseguibile, tutte le disposizioni dei segnali vengono **reimpostate all'azione di default**, eccetto quelle segnate come **ignore**, che vengono mantenute. Gli **handler intercettati** non vengono conservati, perché l'**indirizzo della funzione gestore** non ha più significato nel nuovo contesto di memoria: l'immagine del processo precedente viene **completamente sovrascritta**, inclusa la parte contenente il codice del gestore. Di conseguenza, ogni segnale torna al comportamento predefinito del sistema.

Esempio: Gestore che “cattura” vari segnali e ne stampa il tipo.

```
# include    <signal.h>
# include    "apue.h"
static void  sig_usr(int); // un solo handler per tutti
int main(void)
{
    if (signal(SIGUSR1, sig_usr) == SIG_ERR)
        err_sys("can't catch SIGUSR1");
    if (signal(SIGUSR2, sig_usr) == SIG_ERR)
        err_sys("can't catch SIGUSR2");
    if (signal(SIGINT, sig_usr) == SIG_ERR)    // <CTRL C>
        err_sys("can't catch SIGINT");
    if (signal(SIGTSTP, sig_usr) == SIG_ERR)    // <CTRL Z>
        err_sys("can't catch SIGTSTP");

    for ( ; ; ) pause();
}
```

Funzione:

```
static void
sig_usr(int signo) // signo è il numero del segnale
{
    if (signo == SIGUSR1)
        printf("received SIGUSR1\n");
    else if (signo == SIGUSR2)
        printf("received SIGUSR2\n");
    else if (signo == SIGINT)
        printf("received SIGINT\n");
    else if (signo == SIGTSTP)
        printf("received SIGTSTP\n");
    else
        err_dump("received signal %d\n", signo);
    return;
}
```

L'esempio mostrato illustra l'uso di un **gestore di segnali** in linguaggio C, che permette a un processo di **intercettare diversi segnali** e **stampare informazioni** su di essi al momento della ricezione.

Nel programma, viene definita una funzione `sig_usr(int signo)`, che agisce da **gestore comune** per più segnali. La funzione `signal()` viene chiamata per collegare questo gestore ai segnali **SIGUSR1**, **SIGUSR2**, **SIGINT** (CTRL-C) e **SIGTSTP** (CTRL-Z). Qualora l'associazione fallisca, viene generato un errore tramite la funzione `err_sys()`.

La funzione `sig_usr()` stampa un messaggio identificando il segnale ricevuto. Se il segnale non è tra quelli previsti, viene richiamata `err_dump()` per stampare il numero del segnale non gestito.

Quando l'utente esegue il comando `kill -SIGUSR2 <pid>` da terminale, la shell interpreta **SIGUSR2** come una **macro simbolica** definita in `<signal.h>` e la converte nel relativo **valore intero** (ad esempio 12). La shell invoca quindi la system call `kill(pid, 12)`, richiedendo al **kernel** di inviare il segnale al processo indicato.

Il **kernel** riceve la richiesta, verifica che il processo esista e che l'utente abbia i **permessi** per interagire con esso. Se la richiesta è valida, il kernel rende il segnale **pendente** per il processo. Quando il processo è pronto a riceverlo, il kernel consulta la tabella interna dei **gestori di segnali** del processo, dove sono memorizzate le azioni specificate tramite la funzione `signal()`.

Se per il segnale **SIGUSR2** è stato registrato un **gestore personalizzato**, il kernel esegue quella funzione (`sig_usr`) invece di applicare il comportamento di default. Il parametro `signo` passato alla funzione contiene il **numero del segnale** ricevuto, che può essere confrontato con le macro simboliche (**SIGUSR1**, **SIGUSR2**, ecc.) poiché anch'esse rappresentano interi.

Nel programma, il processo si trova in attesa passiva tramite la funzione `pause()`, che lo sospende finché non riceve un segnale. Alla consegna del segnale, `pause()` viene interrotta e il kernel invoca la funzione `sig_usr` con il valore del segnale. La funzione gestore esegue il codice associato (es. una stampa a video), dopodiché il controllo ritorna al punto in cui era stato sospeso, e il processo torna nuovamente in attesa.

System call `kill()` e `raise()`

```
# include <sys/types.h>

# include <signal.h>

int kill(pid_t pid, int signo);

int raise (int signo);
```

Le chiamate di sistema **kill()** e **raise()** rappresentano strumenti fondamentali per l'invio di **segnali tra processi** in ambiente Unix. Entrambe le funzioni permettono di **generare un segnale**, ma con scopi diversi: `kill()` consente di inviarlo a **processi esterni**, mentre `raise()` lo invia al **processo stesso** che la chiama.

La funzione `kill(pid_t pid, int signo)` invia il segnale identificato da **signo** al processo o ai processi identificati da **pid**. Se l'operazione ha successo, restituisce **0**; in caso di errore, restituisce **-1** e imposta la variabile globale **errno**. I permessi per l'invio dipendono dall'identità del chiamante: il processo deve avere lo stesso **real** o **effective user ID** del destinatario, oppure deve essere eseguito con **privilegi di superutente**.

Il parametro **pid** può assumere diversi significati:

- Se `pid > 0`, il segnale è inviato al processo con quel **ID esatto**.
- Se `pid == 0`, il segnale viene inviato a **tutti i processi nel gruppo di processo** del chiamante, incluso se stesso.
- Se `pid < 0`, il segnale viene inviato a **tutti i processi appartenenti al gruppo di processo** il cui **ID è uguale al valore assoluto** di `pid`.

Secondo lo standard **POSIX.1**, il segnale numero **0** è definito come **segnale nullo**: non provoca alcuna azione sul processo destinatario, ma serve per verificare se il processo **esiste** e se il chiamante ha i **permessi necessari** per inviargli segnali. Se il processo non esiste, `kill()` restituisce **-1**.

La funzione `raise(int signo)` invece è usata per inviare un segnale **al processo stesso**. È funzionalmente equivalente a `kill(getpid(), signo)`, ma con una sintassi più semplice. Non ci sono problemi di **permessi**, poiché il segnale è inviato internamente. Se l'operazione riesce, restituisce **0**, altrimenti un valore diverso da zero, ma **non imposta errno**. Anche qui, il parametro `signo` indica il **tipo di segnale da generare**.

Richiedere un segnale di allarme: Funzione alarm()

La funzione **alarm()** permette a un processo di richiedere l'invio di un **segnale temporizzato**, più precisamente **SIGALRM**, dopo un determinato numero di **secondi**.

Dichiarata in `<unistd.h>`, la funzione ha la seguente firma:

```
#include <unistd.h>

unsigned int alarm (unsigned int seconds)
```

Quando viene chiamata, **istruisce il kernel** affinché invii **SIGALRM** al processo chiamante **trascorsi seconds secondi**. Se una precedente chiamata ad `alarm()` era già stata effettuata, quest'ultima viene **annullata** e sostituita dalla nuova. Passare il valore **0** come argomento ha l'effetto di **annullare** un eventuale allarme già schedato senza impostarne uno nuovo.

La funzione restituisce il **numero di secondi rimanenti** prima dell'invio del precedente **SIGALRM**, oppure **0** se non vi era nessun allarme in sospeso.

È importante notare che, sebbene **SIGALRM** venga inviato dopo almeno `seconds` secondi, in pratica la **tempistica non è garantita con assoluta precisione**, poiché può essere influenzata dallo **scheduler** del sistema.

Questa funzione è spesso utilizzata per implementare **timeout** in applicazioni, permettendo di forzare l'interruzione di operazioni bloccanti o troppo lunghe.

Attendere un segnale: Funzione pause()

La funzione **pause()**, definita in `<unistd.h>`, serve per **sospendere l'esecuzione** del processo chiamante **fino alla ricezione di un segnale**.

La sua dichiarazione è la seguente:

```
#include <unistd.h>

int pause (void)
```

Quando viene invocata, il processo entra in **stato di attesa bloccante**, restando inattivo finché non intercetta un **segnale gestibile**. Il controllo viene restituito **solo dopo l'esecuzione del gestore** associato al segnale ricevuto. A quel punto, `pause()` termina **restituendo -1** e impostando il codice di errore `errno` a **EINTR**, che indica che l'attesa è stata interrotta da un segnale.

Questa funzione è spesso usata in applicazioni dove il processo non deve fare altro che **attendere segnali**, ad esempio per sincronizzarsi con eventi esterni o risposte asincrone.

Funzione sleep()

La funzione **sleep()**, definita in `<unistd.h>`, consente di **sospendere l'esecuzione di un processo** per un intervallo di tempo espresso in **secondi**.

La sua dichiarazione è:

```
#include <unistd.h>

unsigned int sleep(unsigned int seconds)
```

Il processo chiamante viene bloccato fino a quando è trascorso il numero di secondi specificato, oppure riceve un **segnale**, e viene eseguito il relativo gestore.

Al termine della sospensione, `sleep()` restituisce **0**, se il tempo richiesto è trascorso completamente oppure il **numero di secondi rimanenti**, se l'attesa è stata interrotta da un segnale.

È importante notare che, come per la funzione `alarm()`, la **restituzione del controllo** al processo potrebbe non essere immediata e subire **lievi ritardi**, dovuti alle attività in corso nel sistema e alla gestione dello **scheduler**.

Questa funzione è utile per introdurre **pause temporanee** nei programmi, ad esempio per operazioni periodiche o per evitare cicli CPU-intensive.

Funzione nanosleep()

```
#include <unistd.h>

int nanosleep(const struct timespec *req, struct timespec *rem)
```

La funzione **nanosleep()**, definita in `<unistd.h>`, consente di **sospendere l'esecuzione di un processo** per un intervallo di tempo specificato con **precisione al nanosecondo**. È particolarmente utile in contesti in cui è richiesta una **temporizzazione precisa**, come nelle applicazioni **real-time** o nei sistemi che richiedono **sincronizzazione fine**.

```
struct timespec {
    time_t tv_sec;           // secondi
    long   tv_nsec;          // nanosecondi
};
```


Gli argomenti della funzione sono due puntatori a strutture di tipo `struct timespec`, definite in `<time.h>`. Questa struttura contiene due campi: `tv_sec`, che rappresenta i secondi, e `tv_nsec`, che rappresenta i nanosecondi. Il campo `tv_nsec` deve essere compreso tra **0 e 999.999.999**. Il primo parametro (**req**) specifica il tempo desiderato di inattività del processo, mentre il secondo parametro (**rem**) viene utilizzato per memorizzare l'eventuale **tempo rimanente** nel caso in cui l'attesa venga interrotta da un segnale.

Se l'intervallo di tempo si esaurisce completamente senza interruzioni, la funzione restituisce **0**. Se invece l'attesa viene **interrotta da un segnale**, la funzione restituisce **-1** e imposta la variabile globale `errno` a `EINTR`, indicando che il sonno è stato interrotto. In alternativa, `EINVAL` può essere impostato se i valori temporali forniti non sono validi. In caso di interruzione, il tempo residuo non ancora atteso viene salvato nella struttura puntata da `rem`, permettendo eventualmente di **riprendere l'attesa residua** successivamente.

Funzione abort()

```
#include <stdlib.h>
void abort (void)
```

La funzione `abort()`, dichiarata in `<stdlib.h>`, è utilizzata per **terminare immediatamente un processo** inviando a sé stesso il **segnale SIGABRT**. Questo segnale, secondo le convenzioni di sistema, **non dovrebbe essere ignorato** dai processi, poiché rappresenta una terminazione anomala.

Lo standard **POSIX.1** stabilisce che la chiamata ad `abort()` **annulla qualsiasi eventuale disposizione** precedente che prevedesse l'ignoramento o il blocco di `SIGABRT`, assicurandone quindi la corretta ricezione.

È possibile intercettare `SIGABRT` mediante un gestore di segnali, il che consente al processo di eseguire **operazioni di pulizia** prima della terminazione, come la chiusura di file o il rilascio di risorse.

Tuttavia, se il processo non termina all'interno del gestore, lo standard **POSIX.1** impone che, **al ritorno del gestore**, la funzione `abort()` provveda comunque a **terminare forzatamente il processo**.

Segnali inaffidabili

Nelle prime versioni di **UNIX**, la gestione dei **segnali era considerata inaffidabile**. In quei sistemi, i segnali potevano **andare persi**: quando un segnale si verificava, il processo poteva non accorgersene, compromettendo la reattività e la correttezza del comportamento.

Il **controllo sui segnali** era molto limitato: un processo poteva solamente **ignorare un segnale** o tentare di **catturarlo**. Tuttavia, non poteva bloccarlo per conservarne la notifica, rendendo difficile **sincronizzare eventi** asincroni o memorizzare la loro occorrenza per una gestione successiva.

```
int sig_int(); // funzione di gestione del segnale
...
signal(SIGINT, sig_int); // definizione del gestore
...

sig_int()
{
    signal(SIGINT, sig_int); // ridefinizione gestore
    ...                     // elaborazione segnale
}
```

Un problema specifico delle prime implementazioni era che, **ogni volta che un segnale veniva ricevuto**, la **disposizione associata** al segnale tornava a quella di **default**, costringendo il programmatore a ridefinire manualmente il gestore del segnale all'interno della sua stessa esecuzione. Questo meccanismo risultava **inefficiente e soggetto a errori**, perché nel tempo tra la ricezione del segnale e la nuova definizione, potevano andare perse ulteriori segnalazioni.

Il problema delle prime implementazioni

Nelle prime versioni di Unix (anni '70-'80), dopo che un segnale veniva gestito, il sistema riportava automaticamente quel segnale alla sua azione di default.

Quindi il programmatore doveva reimpostare il gestore dentro la funzione stessa che lo gestiva.

Se non lo faceva in tempo, e nel frattempo arrivava un altro segnale uguale, quel segnale veniva perso o eseguita l'azione di default (es. terminazione del processo).

Esempio concreto in C

Versione “vecchio comportamento”

```
#include <signal.h>
#include <stdio.h>
#include <unistd.h>

void gestore(int signum) {
    printf("Ricevuto segnale %d\n", signum);
    // Nelle prime implementazioni bisognava ridefinire:
    signal(SIGINT, gestore); // ← Riassegno il gestore
}

int main() {
    signal(SIGINT, gestore); // Definizione iniziale
    while (1) {
        printf("In esecuzione...\n");
        sleep(2);
    }
}
```

In questo caso, se **non si rimette** `signal(SIGINT, gestore);` dentro la funzione, dopo la prima pressione di Ctrl+C, il programma tornerebbe al comportamento di default → **si chiuderebbe**.

Oggi si usa `sigaction()`, che mantiene automaticamente la definizione del gestore.

Rientranza delle funzioni

La **rientranza delle funzioni** è un concetto essenziale per garantire l'esecuzione sicura di codice nei **gestori di segnali**, soprattutto in ambienti **multithread**. Un gestore può essere eseguito **asincronamente**, interrompendo il programma in qualsiasi punto, esattamente come accade con più **thread** concorrenti.

Una funzione è **rientrante** se può essere chiamata simultaneamente o interrotta e ripresa senza alterare lo stato dei dati.

Questo accade se usa solo **dati locali** o gestisce **dati globali** in modo sicuro, ad esempio creando copie temporanee. Una funzione **non rientrante**, invece, può causare **corruzione di stato** se usata da più contesti senza protezioni, come semafori o disabilitazione delle interruzioni.

Funzioni come `malloc()`, `free()` o quelle della **libreria I/O standard** non sono rientranti e devono essere **evitate nei gestori di segnali**. In caso contrario, si rischiano comportamenti **indefiniti**, specialmente se il segnale interrompe operazioni non atomiche, come modifiche su variabili condivise o sequenze di istruzioni macchina apparentemente semplici, ad esempio `temp += 1`.

Anche nell'uso della **libreria I/O standard** possono emergere seri problemi di **rientranza**, soprattutto quando si lavora con **stream condivisi**, come nel caso delle funzioni `printf()`.

In contesti dove l'interruzione asincrona è possibile (come durante la gestione dei segnali), è **fortemente sconsigliato** usare funzioni non rientranti, in particolare quelle della **libreria standard di I/O**.

Se, ad esempio, il **gestore di un segnale** stampa un messaggio tramite `printf()` proprio mentre il programma principale stava eseguendo una chiamata alla stessa funzione sul **medesimo stream**, il risultato può essere **corruzione dei dati**. Questo accade perché entrambe le funzioni accedono e modificano **strutture dati interne condivise**, ovvero lo stream stesso, senza alcuna protezione. Di conseguenza, il messaggio del gestore e quello del programma possono sovrapporsi, rendendo l'output **incoerente o danneggiato**.

Esempio

```
#include <signal.h>
#include <stdio.h>
struct due_interi {int a, b;} dati;
void signal_handler (int signo){
    printf("%d, %d\n", dati.a, dati.b);
    alarm(1);
}
int main(void){
    static struct due_interi zero = {0,0}, uno = {1,1};
    signal(SIGALRM, signal_handler);
    dati=zero;
    alarm(1);
    while(1)
        {dati = zero; dati = uno;}
}
```

Questo esempio mette in evidenza un classico problema legato alla **rientranza** e alla **sincronizzazione** tra il codice principale e un **gestore di segnali**.

Nel programma, una struttura globale chiamata **dati** contiene due interi, `a` e `b`. All'interno del `main()`, il ciclo `while` modifica ripetutamente il contenuto di `dati` alternando i valori `{0,0}` e `{1,1}`. Parallelamente, ogni secondo viene inviato un segnale **SIGALRM**, il cui gestore stampa il contenuto corrente della struttura `dati`.

In teoria, l'output del programma dovrebbe essere sempre **"0,0"** o **"1,1"**, poiché questi sono gli unici valori che `dati` dovrebbe contenere. Tuttavia, l'output reale può includere combinazioni **incoerenti** come **"1,0"** o **"0,1"**. Questo comportamento è causato dal fatto che su molte architetture l'assegnazione di una struttura come `dati` non è **atomica**, ovvero viene eseguita in più istruzioni separate. Durante queste istruzioni, un segnale può interrompere il processo, e il gestore può leggere un valore **parzialmente aggiornato**.

Se il gestore accede alla struttura proprio mentre il processo principale la sta modificando, può osservare uno stato **transitorio e corrotto**, ad esempio `a = 1` e `b = 0`, che non rappresenta alcuno degli stati validi previsti dal programma.

Su alcune architetture più moderne, dove l'assegnazione della struttura può avvenire **in un'unica istruzione non interrompibile**, questo problema non si manifesta, e l'output risulterà corretto.

Questo esempio dimostra l'importanza di evitare **accessi concorrenti non protetti a dati condivisi** tra codice normale e gestori di segnali. In ambienti dove ciò è inevitabile, è fondamentale **utilizzare tecniche di protezione** come segnali bloccati temporaneamente o accesso mediato da variabili atomiche.

Alcune Funzioni rientranti

Al contrario, alcune funzioni sono progettate per essere **rientranti**, ovvero sicure da eseguire anche in presenza di concorrenza o interruzioni. Nell'ambiente **openBSD 3.0**, ad esempio, funzioni come:

exit(), access(), alarm(), cfgetispeed(), cfgetospeed(), cfsetispeed(), cfsetospeed(), chdir(), chmod(), chown(), close(), creat(), dup(), dup2(), execl(), execve(), fcntl(), fork(), fpathconf(), fstat(), fsync(), getegid(), geteuid(), getgid(), getgroups(), getpgrp(), getpid(), getppid(), getuid(), kill(), link(), lseek(), mkdir(), mkfifo(), open(), pathconf(), pause(), pipe(), raise(), read(), rename(), rmdir(), setgid(), setpgid(), setsid(), setuid(), sigaction(), sigaddset(), sigdelset(), sigemptyset(), sigfillset(), sigismember(), signal(), sigpending(), sigprocmask(), sigsuspend(), sleep(), stat(), sysconf(), tcdrain(), tcflow(), tcflush(), tcgetattr(), tcgetpgrp(), tcsendbreak(), tcsetattr(), tcsetpgrp(), time(), times(), umask(), uname(), unlink(), utime(), wait(), waitpid(), write()

e molte altre della libreria standard **POSIX** sono considerate rientranti. Tali funzioni non utilizzano dati statici condivisi e operano solo su dati passati esplicitamente come parametri, rendendole **sicure anche all'interno dei gestori di segnali**.

Esempio d'uso dei segnali

```
#include<signal.h>
#include<stdlib.h>
static void handler(int);
int main(){
    int i,n;
    n = fork();
    if (n == -1){
        fprintf(stderr,"fork fallita\n");
        exit(1);
    }
    else
        if ( n == 0) { /* processo figlio */
            printf("\n(figlio) il mio process-id e` %d\n",getpid());
            printf("\n(figlio) aspetto 3 secondi e invio un segnale a %d\n",getppid());

            for (i=0; i<3; i++){
                sleep(1);
                printf(".\n");
            }

            kill(getppid(),SIGUSR1);
            printf("\n(figlio) ho finito e muoio\n");
            exit(0);
        }
        else { /* processo padre */
            signal(SIGUSR1,handler);
            printf("\n(padre) il mio process-id %d\n",getpid());
            printf("\n(padre) incomincio le mie operazioni\n");
            for (i = 0; i < 6; i++) {
                sleep(1);
                printf("*\n");
            }
            printf("\n(padre) ora muoio anch'io\n");
            exit(0);
        }
    }
```

Handler:

```
static void handler(int signo)
/* operazione alla ricezione di una kill */

{
    if (signo == SIGUSR1)
        printf("\n (padre) RICEVUTO INTERRUPT \n");
    return;
}
```

Questo esempio mostra un utilizzo **pratico dei segnali** per la comunicazione tra un **processo padre** e un **processo figlio** in ambiente Unix.

All'avvio, il processo chiama **fork()**, creando un duplicato: se `n == 0`, siamo nel processo **figlio**; altrimenti siamo nel **padre**.

Il **figlio**, dopo aver stampato il proprio PID e un messaggio, attende per 3 secondi eseguendo tre cicli `sleep(1)`. Al termine, invia un **segnale SIGUSR1 al padre** usando `kill(getppid(), SIGUSR1)` per notificargli che ha concluso. Dopo aver inviato il segnale, stampa un messaggio di terminazione ed esce.

Il **padre**, prima di iniziare le sue operazioni, imposta un **gestore di segnali** per SIGUSR1 tramite `signal(SIGUSR1, handler)`. Quando riceve SIGUSR1, la funzione `handler()` viene eseguita, la quale stampa semplicemente un messaggio che conferma l'intercettazione del segnale. Dopodiché, il padre prosegue le sue operazioni per altri sei secondi, stampando un asterisco per ogni secondo, e infine termina anche lui. La chiamata `signal(SIGUSR1, handler)`; nel **padre** serve a **preparare il processo padre a ricevere e gestire una notifica** sotto forma di segnale (**SIGUSR1**) che sarà **inviata dal figlio**.

L'esecuzione avviene in parallelo tra i due processi. Il segnale **funziona come notifica asincrona**, permettendo al padre di reagire alla conclusione del figlio. L'esempio evidenzia quindi:

- L'uso di **fork()** per creare un processo figlio.
- La comunicazione asincrona mediante **kill()** e **signal()**.
- L'attivazione di un **gestore personalizzato** per rispondere al segnale.

Questa tecnica è utile nei casi in cui un processo ha bisogno di essere **notificato in modo asincrono** di eventi significativi avvenuti in un altro processo, come la conclusione, un errore o uno stato particolare.

Esempio: critical.c

```
#include <stdio.h>
#include <signal.h>
int main (void) {
    void (*oldHandler) (int); /* vecchio handler*/
    printf ("Posso essere interrotto\n");
    sleep (3);
    oldHandler = signal (SIGINT, SIG_IGN); // Ignora Ctrl-c
    printf ("Ora sono protetto da Control-C\n");
    sleep (3);
    signal (SIGINT, oldHandler); /* Ripristina il vecchio handler */
    printf ("Posso essere interrotto nuovamente\n");
    sleep (3);
    printf ("Ciao!\n");
    return 0;
}
```

Nel programma `critical.c`, il meccanismo che permette di **proteggere un pezzo di codice** da interruzioni come Ctrl-C si basa sull'uso della funzione `signal()` e sulla **gestione esplicita del gestore del segnale SIGINT**.

Con l'istruzione `void (*oldHandler) (int);` si dichiara un puntatore a funzione `oldHandler` che servirà a salvare il gestore di segnale precedente per `SIGINT`.

Quando viene eseguita l'istruzione `oldHandler = signal(SIGINT, SIG_IGN);`, accadono due cose importanti. Innanzitutto, il processo **comunica al kernel** che il segnale `SIGINT` deve essere **ignorato**, quindi se l'utente preme `Ctrl-C`, il programma **non verrà interrotto**. Allo stesso tempo, la funzione `signal()` restituisce il **gestore precedente** del segnale, ovvero il comportamento che era attivo prima di questa modifica. Questo valore viene **salvato nella variabile** `oldHandler`.

Nel caso in cui non fosse stato impostato un gestore personalizzato in precedenza, `oldHandler` conterrà il valore **`SIG_DFL`**, che rappresenta il comportamento **di default**, ovvero la **terminazione del processo** alla ricezione del segnale `SIGINT`.

Dopo aver eseguito la parte di codice che si vuole proteggere, il programma chiama nuovamente `signal()`, questa volta passando `oldHandler` come secondo argomento. In questo modo, il processo **ripristina il comportamento originale**, che era stato temporaneamente sospeso. Se prima `SIGINT` causava l'interruzione, ora lo farà di nuovo.

Questo meccanismo funziona perché la funzione `signal()` **non solo imposta un nuovo gestore**, ma restituisce anche quello **attualmente in uso**, rendendo possibile un ciclo completo di **modifica temporanea e ripristino**.

Dal punto di vista operativo, il processo **diventa insensibile a `Ctrl-C`** solo durante la sezione critica. Appena tale sezione termina, **torna a rispondere ai segnali** come prima. Questa tecnica è molto utile quando si ha la necessità di **proteggere operazioni sensibili** (come aggiornamenti di strutture dati, scritture su file o sezioni non rientranti) da **interruzioni asincrone** che potrebbero lasciare il sistema in uno stato incoerente.

Esecuzione:

```
$ ./critical
Posso essere interrotto
Ora sono protetto da Control-C
Posso essere interrotto nuovamente
Ciao!
$
```

Esempio: limit.c

```
#include <stdio.h>
#include <signal.h>
#include <stdlib.h>
int delay;
void childHandler (int); /* gestore terminazione figlio */
int main (int argc, char *argv[]) {
    int pid;
    signal (SIGCHLD, childHandler); /* Installa il gestore della terminazione figlio */
    pid = fork (); /* duplicazione */
    if (pid == 0) { /* Figlio */
        execvp (argv[2], &argv[2]); /* Esegue il comando */
    }
    else { /* Padre */
        sscanf (argv[1], "%d", &delay); /* delay da riga di comando */
        sleep (delay); /* dorme per delay secondi */
        printf ("Child %d exceeded limit and is being killed\n", pid);
        kill (pid, SIGINT); /* Kill il figlio */
    }
    return 0;
}
```

Handler:

```
void childHandler (int sig) { // Eseguito se il figlio termina
int childPid, childStat;      /* prima del genitore */
childPid = wait (&childStat); /* Accetta il codice di terminazione del figlio */
printf ("Child %d terminated within %d seconds\n", childPid, delay);
exit (/* USCITA con SUCCESSO */ 0);
}
```

Il programma `limit.c` mostra come sia possibile **limitare il tempo di esecuzione** di un comando arbitrario lanciato da un processo.

Limit nsec cmd args

Esegue il comando **cmd** con gli argomenti **args** indicati, dedicandovi al massimo **nsec** secondi.

```
$ limit 3 find / -name filechenonce
find: /root/.links: Permission denied
find: /root/.ssh: Permission denied
...
Child 828 exceeded limit and is being killed
$ limit 1 ls
count.c myexec.c redirect.c critical.c
limit.c limit myfork.c lez7.ps
Child 828 terminated within 1 seconds
$
```

L'obiettivo è far sì che un determinato comando venga eseguito solo per un **massimo numero di secondi**, specificato dall'utente, e che venga **terminato automaticamente** se supera tale limite.

Il meccanismo prevede la **creazione di un processo figlio** tramite `fork()`, che esegue il comando specificato con i suoi argomenti tramite `execvp()`. Contemporaneamente, il **processo padre si mette in attesa** per un intervallo di tempo pari al limite fornito dall'utente. Se allo scadere di questo tempo il figlio non è ancora terminato, il padre lo **termina forzatamente** inviandogli il segnale **SIGINT**.

Per poter gestire correttamente anche il caso in cui il processo figlio termini spontaneamente prima del tempo limite, viene definito un **gestore per il segnale SIGCHLD**, che è generato automaticamente dal kernel ogni volta che un processo figlio termina. Questo gestore, `childHandler()`, viene registrato nel padre tramite `signal(SIGCHLD, childHandler);`.

Il gestore esegue una `wait()` per **raccogliere lo stato di terminazione** del figlio, evitando che il processo rimanga come **zombie**, e stampa un messaggio che riporta l'effettiva durata dell'esecuzione prima della terminazione. In tal caso, il programma termina normalmente con stato di uscita 0.

Se invece il tempo massimo viene superato, il padre stampa un messaggio che informa l'utente che il figlio ha superato il limite ed è stato **ucciso**, e poi invia il segnale **SIGINT** al processo figlio.

Questo esempio dimostra l'uso combinato di **segnali, fork, exec e wait** per implementare un **controllo di tempo sull'esecuzione di comandi**, offrendo un modello utile per la gestione di processi controllati o temporanei in ambienti Unix-like.

Esempio: pulse.c (sospensione e ripresa)

```
#include <signal.h>
#include <stdio.h>
int main (void) {
    int pid1;
    int pid2;
    pid1 = fork ();
    if (pid1 == 0) { /* Primo figlio */
        while (1) { /* Ciclo infinito */
            printf ("pid1 is alive\n");
            sleep (1);
        }
    }
    pid2 = fork ();
    if (pid2 == 0) { /* Secondo figlio */
        while (1) { /* Ciclo infinito */
            printf ("pid2 is alive\n");
            sleep (1);
        }
    }

    /* ... continua ... */
    sleep (2);
    kill (pid1, SIGSTOP); /* Sospende il primo figlio */
    sleep (2);
    kill (pid1, SIGCONT); /* Ripristina il primo figlio */
    sleep (2);
    kill (pid1, SIGINT); /* termina il primo figlio */
    kill (pid2, SIGINT); /* termina il secondo figlio */
    return 0;
}
```

L'esempio `pulse.c` mostra come un processo padre possa **controllare l'esecuzione di due processi figli**, sospendendoli, riprendendoli e infine terminandoli, utilizzando i **segnali Unix**.

Il programma inizia creando due **processi figli** tramite `fork()`. Ciascun figlio entra in un **ciclo infinito**, durante il quale stampa ogni secondo un messaggio per indicare che è in esecuzione. Questo comportamento continua in parallelo, a meno che non intervenga il padre.

Il processo padre, una volta creati i due figli, attende per **due secondi**, poi invia un **segnale SIGSTOP** al primo figlio. Questo segnale provoca la **sospensione** immediata del processo, che smette di essere schedato dalla CPU. Durante la sospensione del primo figlio, il secondo figlio continua regolarmente a eseguire il proprio ciclo, e quindi a stampare i suoi messaggi.

Dopo altri **due secondi**, il padre invia il **segnale SIGCONT** al primo figlio, che riprende immediatamente l'esecuzione dal punto in cui era stato sospeso. Ancora due secondi dopo, il padre invia **SIGINT a entrambi i figli**, terminando così definitivamente la loro esecuzione.

Questo esempio è particolarmente utile per mostrare l'uso dei segnali **SIGSTOP**, **SIGCONT** e **SIGINT** come strumenti di **controllo dell'esecuzione dei processi**. Inoltre, l'output prodotto evidenzia chiaramente come la sospensione e la ripresa agiscano solo su uno dei due figli, consentendo all'altro di continuare in autonomia. Si tratta quindi di una dimostrazione concreta di **gestione concorrente dei processi**, con un padre che assume il ruolo di supervisore temporale.

Segnali affidabili

Il concetto di **segnali affidabili** nasce dalla necessità di gestire correttamente situazioni in cui l'arrivo di **segnali multipli** può causare **comportamenti indesiderati**, soprattutto durante l'esecuzione di un **gestore di segnali**. Uno dei problemi più delicati che possono sorgere è l'arrivo di un **secondo segnale** mentre è ancora in corso la gestione del primo. Questo secondo segnale può appartenere a una tipologia diversa, ma in alcuni casi può anche essere dello **stesso tipo**, creando così una situazione potenzialmente pericolosa per l'integrità del programma.

Questo fenomeno può causare **race condition**, ovvero condizioni in cui il comportamento del programma dipende dall'ordine di esecuzione non prevedibile tra più eventi concorrenti. Per questo motivo è essenziale prendere **precauzioni all'interno del gestore**, in modo da evitare interferenze o modifiche impreviste allo stato del processo.

Nei sistemi Unix più evoluti sono disponibili **meccanismi di blocco temporaneo dei segnali**, che consentono di **posticipare la loro gestione** fino al termine della funzione attualmente in esecuzione. Queste caratteristiche permettono di rendere la gestione dei segnali **più sicura e prevedibile**, migliorando l'affidabilità generale del programma, in particolare in ambienti concorrenti o multithread.

Maschera dei segnali: sigprocmask()

Nel contesto della gestione dei **segnali** in ambiente POSIX, la **maschera dei segnali** rappresenta un meccanismo fondamentale per controllare quali segnali possano essere ricevuti da un processo in un determinato momento.

```
#include<signal.h>
int sigprocmask(int how, const sigset_t *set, sigset_t *oset);
```

Lo strumento principale per tale scopo è la funzione **sigprocmask()**, definita in `<signal.h>`, che permette di **bloccare temporaneamente** un insieme di segnali, impedendone l'elaborazione durante l'esecuzione di **sezioni critiche** del codice, e di **ripristinare** successivamente la maschera precedente.

La funzione ha la seguente firma: `int sigprocmask(int how, const sigset_t *set, sigset_t *oset);`. L'argomento `how` stabilisce l'operazione da compiere:

- **SIG_BLOCK** per aggiungere segnali alla maschera corrente,
- **SIG_UNBLOCK** per rimuoverli,
- **SIG_SETMASK** per sostituire completamente la maschera.

L'argomento `set` rappresenta l'insieme dei segnali su cui intervenire, mentre `oset`, se non è NULL, consente di salvare la **maschera precedente** per un eventuale ripristino.

Se durante il blocco un segnale viene inviato, esso non viene perso, ma **rimane pendente** e sarà consegnato non appena verrà sbloccato.

È importante notare che se un segnale **pendente** viene sbloccato tramite `sigprocmask()`, il kernel **lo consegnerà immediatamente** al processo, appena l'invocazione della funzione terminerà. Questo comportamento garantisce che i segnali non vengano persi, ma solo **differiti** nel tempo.

Questo comportamento assicura una gestione affidabile e ordinata dei segnali asincroni, evitando **condizioni di competizione** e mantenendo la **correttezza dell'esecuzione concorrente**.

Sebbene in teoria ogni segnale potrebbe essere rappresentato da **un singolo bit di un intero**, il numero di segnali gestibili in un sistema operativo moderno può **eccedere la dimensione di un intero standard**. Per questo motivo, l'uso di un semplice `int` non sarebbe sufficiente o affidabile.

Per risolvere questo problema, POSIX introduce il tipo **sigset_t**, una struttura appositamente progettata per contenere **set di segnali** in modo efficiente.

Grazie a `sigset_t`, è possibile **manipolare insiemi arbitrari di segnali**, indipendentemente dal loro numero, garantendo portabilità e correttezza nell'uso delle API POSIX per la gestione dei segnali.

Il tipo di dato utilizzato per rappresentare gli insiemi di segnali è **sigset_t**, progettato per essere portabile e in grado di contenere un numero arbitrario di segnali, a differenza di un semplice intero.

```
#include<signal.h>

int sigemptyset(sigset_t *set);
int sigfillset(sigset_t *set);
int sigaddset(sigset_t *set, int signo);
int sigdelset(sigset_t *set, int signo);
    Ritornano 0 se OK, -1 in caso di errore

int sigismember(const sigset_t *set, int signo);
    Ritorna 1 se vero, 0 se falso, -1 in caso di errore
```

Per la manipolazione di oggetti di tipo **sigset_t**, POSIX mette a disposizione un insieme di funzioni ausiliarie:

- **sigemptyset()** inizializza un insieme vuoto,
- **sigfillset()** lo riempie con tutti i segnali disponibili,
- **sigaddset()** aggiunge un segnale specifico,
- **sigdelset()** lo rimuove,
- **sigismember()** verifica se un segnale appartiene all'insieme, restituendo 1 in caso positivo, 0 in caso negativo, e -1 in caso di errore.

Tutte queste funzioni, ad eccezione di **sigismember**, restituiscono 0 in caso di successo e -1 in caso di errore, consentendo una gestione robusta degli errori. L'uso combinato di **sigprocmask()** e delle funzioni per la manipolazione dei set di segnali rappresenta una **pratica essenziale** per la protezione di sezioni di codice critico e per garantire la **sicurezza e stabilità** dei programmi concorrenti.

Esempi

```
/* definisce una nuova maschera */
sigset_t mask_set;
/* svuota la maschera */
sigemptyset(&mask_set);
/* aggiunge i segnali TSTP e INT alla maschera */
sigaddset(&mask_set, SIGTSTP);
sigaddset(&mask_set, SIGINT);
/* rimuove il segnale TSTP dall'insieme */
sigdelset(&mask_set, SIGTSTP);
/* controlla se il segnale INT è definito nell'insieme */
if (sigismember(&mask_set, SIGINT))
    printf("segnale INT nell'insieme\n");
else
    printf("segnale INT non nell'insieme\n");
/* pone tutti i segnali del sistema nell'insieme */
sigfillset(&mask_set)
```

Questo frammento di codice mostra un uso pratico delle **funzioni per la gestione degli insiemi di segnali**, tramite il tipo **sigset_t**, e serve a illustrare le principali operazioni che si possono compiere su una **maschera di segnali**.

Viene prima definita una nuova variabile di tipo **sigset_t**, chiamata **mask_set**, che rappresenta un **insieme di segnali**. Inizialmente questo insieme viene **svuotato** tramite **sigemptyset()**, così da partire da una maschera priva di segnali.

Successivamente, vengono **aggiunti due segnali** specifici: SIGTSTP (sospensione da terminale, solitamente generata con Ctrl-Z) e SIGINT (interruzione da terminale, ad esempio Ctrl-C). Questi vengono inseriti con due chiamate a `sigaddset()`.

Poi si procede a **rimuovere** SIGTSTP dal set utilizzando `sigdelset()`, lasciando quindi solo SIGINT nel set. A questo punto, si effettua un **controllo logico** tramite `sigismember()` per verificare se SIGINT è effettivamente presente nell'insieme, e si stampa un messaggio diverso a seconda del risultato.

Infine, l'intero set viene **riempito con tutti i segnali disponibili** usando `sigfillset()`. Questa operazione sovrascrive il contenuto precedente del set, rendendolo pieno.

Questo esempio illustra chiaramente come costruire, modificare e interrogare una **maschera di segnali**, che può poi essere utilizzata in chiamate come `sigprocmask()` per **bloccare, sbloccare o mascherare segnali** nel contesto di un processo.

Esempio

- Vediamo un breve esempio (pezzi) di codice che conta il numero di segnali Ctrl-C digitati dall'utente

- la quinta volta chiede all'utente se vuole terminare
- se l'utente digita Ctrl-Z è stampato il numero di Ctrl-C digitati

```
/* Definiamo il contatore di Ctrl-C counter inizializzato a 0 */
int ctrl_c_count = 0;
#define CTRL_C_THRESHOLD 5
void catch_int(int sig_num) { /* gestore segnale Ctrl-C */
    sigset_t mask_set; /* per impostare la maschera dei segnali */
    sigset_t old_set; /* memorizza la vecchia maschera */
    /* maschera gli altri segnali mentre siamo nel gestore*/
    sigfillset(&mask_set);
    sigprocmask(SIG_SETMASK, &mask_set, &old_set);

    ctrl_c_count++; /* incrementa count, e controlla la soglia */
    if (ctrl_c_count >= CTRL_C_THRESHOLD) {
        char answer[30];
        printf("\nVuoi terminare? [s/N]: ");
        fflush(stdout);
        gets(answer);
        if (answer[0] == 's' || answer[0] == 'S') {
            printf("\nTermino...\n");
            fflush(stdout);
            exit(0); }
        else {
            printf("\nContinuo\n");
            fflush(stdout);
            /* reset del contatore per Ctrl-C */
            ctrl_c_count = 0; }
    }
}
```

```

void catch_suspend(int sig_num) {
    sigset_t mask_set; /* usato per mascherare i segnali */
    sigset_t old_set; /* memorizza la vecchia maschera */
    sigfillset(&mask_set);
    sigprocmask(SIG_SETMASK, &mask_set, &old_set);
    /* stampa il contatore corrente per Ctrl-C */
    printf("\n\n'%d' Ctrl-C digitazioni\n", ctrl_c_count);
    fflush(stdout);
}

/* da qualche parte nel main... */ . .
/* imposta i gestori per the Ctrl-C e Ctrl-Z */
signal(SIGINT, catch_int);
signal(SIGTSTP, catch_suspend);

. . /* ed il resto del programma */ . .

```

Questo esempio di codice mostra come gestire in modo controllato i segnali inviati da tastiera, in particolare **SIGINT** (generato da Ctrl-C) e **SIGTSTP** (generato da Ctrl-Z), utilizzando i meccanismi POSIX per la gestione e la mascheratura dei segnali.

Il programma definisce un **contatore** chiamato `ctrl_c_count`, inizializzato a zero, che serve per registrare quante volte l'utente ha premuto Ctrl-C. Alla quinta pressione (soglia definita da `CTRL_C_THRESHOLD`), viene chiesto all'utente se desidera terminare il programma. In base alla risposta (s o S per sì), il processo viene interrotto; in caso contrario, il contatore viene **azzerato** e il programma continua.

Quando si riceve un segnale, ad esempio SIGINT, viene invocata la funzione **gestore** `catch_int`. In questa funzione, il gestore **maschera temporaneamente tutti gli altri segnali** usando `sigfillset` e `sigprocmask`, in modo da evitare che altri segnali interferiscano mentre si esegue il codice critico, come la lettura dell'input o l'aggiornamento del contatore. Alla fine della gestione, la **maschera dei segnali viene ripristinata** al valore precedente.

Il segnale **SIGTSTP** è gestito dalla funzione `catch_suspend`, che ha la sola funzione di stampare il valore attuale del contatore `ctrl_c_count`, anche in questo caso **proteggendo la stampa con una maschera**.

Infine, nel `main` vengono installati i gestori tramite la funzione `signal()` per collegare **SIGINT** a `catch_int` e **SIGTSTP** a `catch_suspend`.

In sintesi, l'esempio dimostra come implementare un **comportamento personalizzato e protetto** in risposta a segnali di interruzione dell'utente, utilizzando strutture dati come `sigset_t`, funzioni di manipolazione della maschera dei segnali come `sigprocmask`, e l'uso di **gestori di segnale** per proteggere porzioni di codice sensibili.

Evitare le race condition: sigaction()

L'uso della sola funzione **sigprocmask()** non è sufficiente per evitare tutte le possibili **race condition** nella gestione dei segnali. In particolare, può verificarsi una situazione in cui un **segnale** arriva **tra l'entrata nel gestore** e la successiva chiamata a `sigprocmask()`. Se ciò accade, il **nuovo segnale viene gestito immediatamente**, causando un possibile conflitto con l'esecuzione del gestore già attivo.

```

#include<signal.h>

int sigaction(int signo, const struct sigaction *act, struct sigaction *oact);

```

Per ovviare a questo problema, il sistema POSIX fornisce la funzione **sigaction()**, che rappresenta una soluzione più sicura e avanzata per associare un **gestore a un segnale**, specificando contestualmente una **maschera temporanea** di segnali da bloccare durante la sua esecuzione. In questo modo, si impedisce che un secondo segnale venga gestito **mentre il primo è ancora in corso**, evitando sovrapposizioni e comportamenti non deterministici.

La funzione `sigaction(int signo, const struct sigaction *act, struct sigaction *oact)` è **rientrante**, e sostituisce `signal()` per la sua maggiore flessibilità e sicurezza. Restituisce 0 in caso di successo e -1 in caso di errore, impostando `errno`. Il parametro `signo` identifica il **segnale da gestire**, mentre `act`, se non nullo, punta alla **nuova azione da associare**; `oact`, anch'esso opzionale, può essere usato per **salvare la vecchia azione**.

```
struct sigaction {
    void (*sa_handler)(int);
    sigset_t sa_mask;
    int sa_flags;
    void (*sa_sigaction)(int, siginfo_t *, void *);
};
```

La struttura `sigaction` include diversi campi. Il campo **`sa_handler`** è un puntatore alla funzione gestore (oppure può essere `SIG_IGN` o `SIG_DFL` per ignorare o ripristinare il comportamento predefinito). In alternativa, il campo **`sa_sigaction`** consente di usare un gestore più avanzato che riceve informazioni dettagliate tramite una struttura `siginfo_t`, oltre al contesto di esecuzione. Il campo **`sa_mask`** specifica l'insieme di segnali da bloccare **durante l'esecuzione del gestore** (incluso automaticamente `signo`), che viene poi **ripristinato automaticamente** al termine del gestore, garantendo così isolamento e ordine nell'elaborazione dei segnali.

Il campo **`sa_flags`** permette di definire ulteriori comportamenti: `SA_SIGINFO` abilita l'uso del gestore avanzato; `SA_RESTART` fa in modo che le **system call interrotte da un segnale vengano automaticamente riavviate**, evitando la necessità di controllare `errno` per `EINTR` e rilanciare la chiamata manualmente; `SA_NODEFER` consente al segnale corrente di non essere bloccato durante la sua gestione, permettendo così **ri-entrate**; infine, `SA_NOCLDWAIT` impedisce la creazione di **processi zombie** alla terminazione dei figli.

Grazie a queste caratteristiche, `sigaction()` consente una gestione **robusta, sicura e portabile** dei segnali, ed è particolarmente utile in ambienti **concorrenti o asincroni**, dove l'affidabilità del gestore e l'isolamento degli eventi sono essenziali.

Esempio

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>

void signal_handler(int signum) {           // Gestore dei segnali
    printf("Ricevuto il segnale %d\n", signum);
}

int main() {
    struct sigaction sa;
    sa.sa_handler = signal_handler; // Impostazione del gestore dei segnali
    sa.sa_flags = 0;
    sigemptyset(&sa.sa_mask);
    if (sigaction(SIGINT, &sa, NULL) == -1) { // Installazione del gestore per il segnale SIGINT (Ctrl+C)
        perror("Errore nell'installazione del gestore del segnale");
        exit(EXIT_FAILURE);
    } ...
}
```

Questo esempio mostra l'utilizzo della system call **`sigaction()`** per installare un **gestore di segnali personalizzato** per il segnale **`SIGINT`**, ovvero quello generato dalla pressione di **`Ctrl+C`**.

Nel `main`, si definisce una struttura **`sigaction`**, che viene inizializzata impostando il campo **`sa_handler`** alla funzione `signal_handler`, cioè la funzione che verrà chiamata quando il segnale sarà ricevuto. Il campo **`sa_flags`** viene posto a zero,

mentre **sa_mask** viene svuotato con `sigemptyset()`, il che significa che **nessun altro segnale viene bloccato** durante l'esecuzione del gestore.

Successivamente, viene chiamata **sigaction()** per registrare la struttura come gestore del segnale **SIGINT**. Se la chiamata fallisce, viene stampato un messaggio di errore tramite `error()` e il programma termina con `EXIT_FAILURE`.

Grazie a questa configurazione, quando l'utente preme **Ctrl+C**, il programma **non viene terminato**, ma invece viene eseguita la funzione `signal_handler()`, che stampa il numero del segnale ricevuto. Questo dimostra come sia possibile **intercettare e gestire segnali asincroni in modo controllato e sicuro**.

Segnali per il controllo dei job

Gruppi di processi

I segnali sono strumenti fondamentali per il controllo dei job nei sistemi Unix-like. Ogni **processo** è parte di un **gruppo di processi**, identificato tramite un **process group ID (pgid)**. Questo identificatore è distinto dal **GID** (group ID) usato per la gestione dei permessi. Un **processo figlio** eredita automaticamente il gruppo del padre, ma può modificarlo usando la system call **setpgid()**, che consente anche il cambiamento del gruppo di appartenenza di altri processi.

A ogni processo può essere associato un **terminale di controllo**, di solito quello da cui è stato lanciato. I figli ereditano il terminale di controllo del padre. Se un processo esegue una **exec()** il terminale di controllo non cambia. Parallelamente, ogni terminale ha associato un **processo di controllo**, responsabile della gestione dei segnali interattivi.

Ad ogni terminale è associato un **processo di controllo**. Se il terminale individua un metacarattere come **Ctrl-C**, spedisce il segnale appropriato a tutti i processi nel gruppo del processo di controllo. Se un processo tenta di leggere dal suo terminale di controllo e non è membro del gruppo del processo di controllo di quel terminale, il processo riceve un segnale **SIGTTIN** che, normalmente, lo sospende (**SIGTTOU** se tenta di scrivere sul terminale di controllo).

Nel contesto di una **shell interattiva**, questo meccanismo si evidenzia in modo chiaro. Quando la shell esegue un **comando in foreground**, la shell figlio si mette in un diverso gruppo, assume il controllo del terminale, esegue il comando. Così ogni segnale generato dal terminale viene indirizzato al comando e non alla shell originaria. Quando il comando termina, la shell originaria riprende il controllo del terminale. quest'ultimo viene messo in un gruppo distinto e prende il controllo del terminale. Invece, se la shell esegue un **comando in background**, la shell figlio si mette in un diverso gruppo ed esegue il comando, ma non assume il controllo del terminale. Così ogni segnale generato dal terminale continua ad essere indirizzato alla shell originaria. Se il comando in background tenta di leggere dal suo terminale di controllo, viene sospeso da un segnale **SIGTTIN**.

Questo comportamento consente una **gestione fine del multitasking** e delle interazioni utente, fornendo un robusto meccanismo per il controllo dei job in ambienti interattivi.

Cambiare il gruppo: setpgid()

```
int setpgid(pid_t pid, pid_t pgrpId)
```

La funzione **setpgid(pid, pgrpId)** serve ad assegnare al processo identificato da **pid** un nuovo gruppo di processi, il cui identificatore è **pgrpId**. Se **pid** è 0, si modifica il gruppo del processo chiamante. Se **pgrpId** è 0, al processo viene assegnato come gruppo il proprio **PID**. Questa funzione ha successo solo se il processo chiamante ha gli stessi identificatori utente (**uid/gid**) di quello indicato da **pid**, oppure se è il **superuser**. Se la chiamata ha successo, restituisce **0**, altrimenti **-1**.

Questa funzione è utile, ad esempio, quando un processo vuole creare un **nuovo gruppo di processi**: in tal caso può semplicemente invocare `setpgid(0, getpid())`, cioè crea un gruppo con sé stesso come leader.

Ottenere il gruppo: getpgid()

```
pid_t getpgid(pid_t pid)
```

La funzione **getpgid(pid)**, invece, restituisce l'ID del gruppo di processi a cui appartiene il processo indicato da **pid**. Se **pid** è 0, ritorna il gruppo del processo chiamante. A differenza di **setpgid()**, questa funzione **non fallisce mai**.

Esempio: pgrp1.c

```
#include <signal.h>
#include <stdio.h>
void sigHandler (int sig) {
    printf ("Process %d got a %d signal\n", getpid(), sig);
}
int main (void) {
    signal (SIGINT, sigHandler); /* Gestisce Control-C */
    if (fork () == 0)
        printf ("Child PID %d PGRP %d waits\n",getpid(),getpgid(0));
    else
        printf ("Parent PID %d PGRP %d waits\n",getpid(),getpgid(0));
    pause (); /* Aspetta un segnale */
}
```

L'esempio mostrato riguarda la relazione tra **gruppi di processi (process group)** e **segnali generati da terminale**, come ad esempio **Ctrl-C**, che invia il segnale **SIGINT**.

Nel programma **pgrp1.c**, il processo principale crea un **gestore di segnali** per **SIGINT**, cioè **sigHandler**, e successivamente esegue una **fork()**, generando così un processo **figlio**. Entrambi i processi — genitore e figlio — stampano il loro **PID** (process ID) e il **PGRP** (process group ID) a cui appartengono. Infine, chiamano **pause()** per rimanere in attesa di segnali.

```
$ ./pgrp1
Parent PID 24444 PGRP 24444 waits

Child PID 24445 PGRP 24444 waits

<^C> Process 24445 got a 2 signal

Process 24444 got a 2 signal
$
```

La cosa interessante avviene quando l'utente preme **Ctrl-C** nel terminale: questo genera un **SIGINT** che viene inviato **a tutti i processi del gruppo di controllo del terminale**, ovvero a tutti i processi che condividono lo stesso **process group ID** del processo in foreground.

Poiché sia il processo padre che il figlio condividono lo **stesso gruppo di processi**, entrambi ricevono il segnale **SIGINT**, che viene gestito dalla funzione **sigHandler**. L'output mostra chiaramente che entrambi i processi ricevono il segnale 2 (che corrisponde a **SIGINT**), confermando che **il segnale viene distribuito all'intero gruppo di processi**.

Questo comportamento è alla base del **job control** nelle shell Unix: la shell può inviare segnali a gruppi di processi interi per sospendere, continuare o terminare un intero job.

Esempio: pgrp2.c

```
void sigHandler (int sig) {
printf ("Process %d got a SIGINT\n", getpid());
exit (1);
}
int main (void) {
int i;
signal (SIGINT, sigHandler); /* gestore di segnali */
if (fork () == 0)
setpgid (0, getpid ()); /* il figlio nel suo gruppo */
printf("Process PID %d PGRP %d waits\n",getpid(),getpgid(0));
for (i = 1; i <= 3; i++) { /* Cicla 3 volte */
printf ("Process %d is alive\n", getpid());
sleep(2);
}
return 0;
}
```

Il programma **pgrp2.c** dimostra come il comportamento dei segnali cambi a seconda del **gruppo di processi** a cui appartiene un processo. In particolare, evidenzia che un processo che **lascia il gruppo del processo di controllo del terminale** non riceverà più i segnali inviati dal terminale stesso, come il **Ctrl-C (SIGINT)**.

All'avvio, entrambi i processi, padre e figlio, ereditano lo stesso **gruppo di processi**. Tuttavia, il figlio viene subito spostato in un **nuovo gruppo** usando la system call **setpgid()**, con il proprio PID come nuovo **process group ID**. Questo lo separa dal gruppo di controllo del terminale.

Nel codice, entrambi i processi installano lo stesso **gestore del segnale SIGINT**, che semplicemente stampa un messaggio e termina il processo. Dopo aver eseguito **fork()**, i due processi iniziano a stampare un messaggio ogni due secondi.

```
• esecuzione...
$ ./pgrp2
Process PID 24535 PGRP 24535 waits

Process 24535 is alive

Process PID 24536 PGRP 24536 waits

Process 24536 is alive

<^C> Process 24535 got a SIGINT

$ Process 24536 is alive

Process 24536 is alive

<return>
$
```


Quando l'utente preme **Ctrl-C**, il terminale invia il **SIGINT** a tutti i processi nel **gruppo di controllo del terminale**, che è ancora il gruppo del processo padre. Di conseguenza, **solo il processo padre riceve e gestisce il segnale**, mentre il figlio, essendo in un gruppo separato, **non viene coinvolto**. Questo comportamento è esattamente quello che il programma vuole illustrare.